UltraGS: руководство программиста

Русское руководство к текущему API 1.0 / VLS1. Для программиста Sprinter, знакомого с Z80. Редакция 4, 23 октября 2026 года.

Руководство объясняет работу с картой с самого начала. Достаточно уметь читать простые команды Z80: `LD`, `CALL`, `RET` и переходы. Термины звукового API вводятся по мере использования. Начните с глав 1 и 2; таблицы в конце нужны, когда вы будете писать собственную программу. Подготовка карты и сборка исходников описаны в README.

Оглавление

1. Что программа поручает UltraGS 3
   1.1. Какие звуки можно воспроизводить . . . 3
   1.2. Какой интерфейс выбрать . . . 3
2. Первый звук: низкий тон из собственного массива 4
   2.1. Обратиться к нужной карте . . . 4
   2.2. Подготовить числа, которые станут звуком . . . 5
   2.3. Попросить карту выделить память . . . 5
   2.4. Скопировать массив на карту . . . 6
   2.5. Сообщить карте, как читать массив . . . 6
   2.6. Включить звук . . . 7
   2.7. Остановить звук и вернуть память . . . 8
   2.8. Дополнительная проверка: прочитать свои байты обратно . . . 8
   2.9. Вся последовательность и первые изменения . . . 9
3. Интерфейс со Sprinter 10
   3.1. Адреса и ISA-окно . . . 10
   3.2. Командный и информационный флаги GS . . . 10
   3.3. Расширенные запросы и ответы . . . 11
   3.4. ABI библиотеки . . . 11
4. Система команд GS 1.x 12
   4.1. Как применять таблицу . . . 14
   4.2. Загрузка MOD и FX . . . 15
   4.3. Отличия, которые влияют на программу . . . 15
   4.4. Остальные исторические команды . . . 16
5. Расширенный UltraGS API 1.0 17
   5.1. Обнаружение и память . . . 17
   5.2. Загрузка и чтение . . . 17
   5.3. MOD, PCM и FX . . . 17
   5.4. Streaming . . . 18
   5.5. Снимки, статистика и ошибки . . . 19
6. Память, запуски и завершение программы 20
   6.1. Три разных номера: объект, запуск, голос . . . 20
   6.2. Зачем существует session и когда о ней нужно знать . . . 20
   6.3. Что освобождать при выходе . . . 21
   6.4. Музыка, эффекты и FM вместе . . . 21
7. AdLib OPL2 22
8. MOD, эффекты, FM и поток: примеры кода 23
   8.1. Загрузить MOD через команды GS . . . 23
   8.2. Загрузить и запустить GS FX . . . 24
   8.3. Запустить MOD8 и добавить mono/stereo FX . . . 25
   8.4. Добавить FM поверх MOD . . . 27
   8.5. Подавать WAV с диска небольшими блоками . . . 29
9. Приложения 31
   9.1. Карта портов и биты возможностей . . . 31
   9.2. Ошибки и действия программиста . . . 33
   9.3. Утилиты . . . 34
   9.4. Совместимость и расхождения источников . . . 34
   9.5. Словарь . . . 35
   9.6. Источники и авторы . . . 35
   9.7. Форматы команд UltraGS API 1.0 . . . 35
   9.8. Вложенные структуры и записи . . . 52

1. Что программа поручает UltraGS

UltraGS - звуковая карта для Sprinter, точнее режим работы платы PicoGUS. Программа на Z80 передаёт ей данные и команды. После команды воспроизведения карта сама формирует звук; Z80 может в это время рисовать экран, читать клавиатуру или выполнять другую работу.

Начнём с самого простого вида звука: последовательности чисел, описывающих форму звуковой волны. Каждое число называется **отсчётом**. Например, карта может брать из массива 8000 отсчётов в секунду. Такая запись звука называется **PCM**. Сам массив может содержать речь, удар, музыкальную ноту или любой другой записанный звук.

Работа программы состоит из трёх частей:

1. Передать массив в память карты и сообщить, как его читать.
2. Дать команду воспроизведения. Карта вернёт номер этого запуска.
3. Когда звук больше не нужен, остановить запуск и удалить массив из памяти карты.

Память Z80 и память карты различаются. Массив в вашей программе сначала лежит в памяти Z80. Пока программа не скопировала его на карту, проиграть его там нельзя. Сохранённый на карте массив называется **объектом**. Карта выдаёт ему номер, **handle**: по этому номеру программа впоследствии выбирает звук. Номер не является адресом памяти и не задаётся программистом заранее.

1.1. Какие звуки можно воспроизводить

Вид звука	Что передаёт программа	Для чего удобно
RAW PCM	Массив отсчётов; частоту и формат задаёт отдельной командой	Короткие эффекты и собственный генератор звука
WAV PCM	Файл с отсчётами и заголовком, содержащим частоту и формат	Записанная речь, звуки и музыка
MOD	Модуль: образцы инструментов и последовательность нот	Фоновая музыка без постоянной работы проигрывателя на Z80
OPL2 / FM	Настройки синтезатора и команды включения нот	Синтез звука без загрузки массива отсчётов

Mono содержит один звуковой канал; **stereo** - левый и правый. Для каждого одновременно воспроизводимого PCM-канала карта выделяет один **голос**. Всего таких голосов 32: mono-эффект занимает один, stereo-эффект - два, восьмиканальный MOD - восемь. FM-синтезатор OPL2 работает отдельно и эти голоса не занимает.

На PicoGUS 2.0 установлено 8 МиБ памяти для данных. Она хранит загруженные звуки до их удаления или сброса карты. Её нельзя читать обычной командой `LD` по адресу Z80: чтение и запись выполняются через библиотеку. О поддержанных разновидностях WAV/MOD и ограничениях памяти сказано в главах 5 и 6; полный список форматов приведён в приложении 9.4.

1.2. Какой интерфейс выбрать

Для новой программы используйте **расширенный UltraGS API**. API здесь означает набор команд, которые понимает карта: загрузить звук, запустить, приостановить, прочитать состояние. В главе 2 показана работа через готовую библиотеку Z80. Вам не нужно сразу писать собственный обмен через порты.

Команды GS нужны прежде всего для программ, уже рассчитанных на General Sound. Они описаны отдельно в главе 4. UltraGS воспроизводит музыку по этим командам, но не исполняет внутри карты произвольные Z80-программы для GS или NeoGS. Внутренняя RAM GS, POKE/PEEK/CALL, MP3-декодер и SD-интерфейс NeoGS здесь не предоставляются.

2. Первый звук: низкий тон из собственного массива

Задача: получить короткий гудок без WAV-файла и без музыкального модуля. Программа создаст массив из 256 байт, загрузит его в карту, будет повторять примерно 1-2 секунды, затем остановит звук и освободит память. ESC позволяет закончить раньше. При успешном выходе загруженный массив на карте не остаётся.

Все фрагменты ниже взяты из firstpcm.asm. Они являются частями одной программы, а не отдельными EXE. Библиотечные процедуры начинаются с `UG_`; остальные, например `upload_sound`, написаны в самом примере и разобраны ниже. Полный файл содержит также заголовок EXE, общий выход и обработку ошибок. DSS - операционная система Sprinter, через которую пример читает клавиатуру и возвращается в командную строку.

Карта должна быть настроена в режим UltraGS. Для этого учебного примера её память должна быть свободна, а других программ, управляющих картой, быть не должно. Если уже загружены чужие звуки, пример сообщит BUSY («занято») и завершится. В дальнейшем можно работать и с несколькими звуками.

2.1. Обратиться к нужной карте

Сначала библиотеке нужно сообщить **где находится карта**. Здесь два независимых параметра:

Параметр	Значение в примере	Что оно означает
Номер слота	`DOC_SLOT`, по умолчанию 0	В какой ISA-слот Sprinter установлена карта: 0 или 1
Базовый адрес портов	`CARD_IO_BASE`, равный `02A0h`	С какого номера начинаются порты UltraGS на шине ISA

Порт - адрес, по которому компьютер обращается к регистру устройства: передаёт команду или читает ответ. Карта занимает несколько соседних портов; адрес первого из них называют **базовым**, или BASE. `02A0h` - стандартная настройка UltraGS. Текущий адрес показывает утилита `UGCFG INFO`. Это настройка самой карты: команда `UGCFG PORTS` позволяет изменить её без перепрошивки, а `UGCFG SAVE` сохраняет после выключения питания. Порядок настройки приведён в README. В `UG_OPEN` передают уже установленный адрес: этот вызов не перенастраивает карту. Если BASE не меняли, оставьте `02A0h`.

`0x02A0` и `02A0h` - две записи одного шестнадцатеричного числа. В исходнике оно записано один раз под понятным именем:

```
CARD_IO_BASE EQU 0x02A0       ; первый порт UltraGS на шине ISA
```

Далее передаём библиотеке номер слота через регистр A, а адрес портов через пару HL:

```
    call UG_LIB_INIT          ; подготовить переменные библиотеки
    ld a,DOC_SLOT             ; слот с картой: 0 или 1
    ld hl,CARD_IO_BASE        ; адрес первого порта карты
    call UG_OPEN              ; проверить, что карта отвечает
    ret c                     ; при ошибке вернуться к общему выходу
```

`UG_LIB_INIT` подготавливает переменные библиотеки в памяти Z80. `UG_OPEN` проверяет ответ карты и запоминает, куда обращаться при следующих вызовах. **Звука после этого ещё нет:** массив пока не загружен и команда воспроизведения не дана.

После библиотечного вызова установленный флаг переноса C означает ошибку; её код находится в A. Поэтому `RET C` возвращается из текущего шага при ошибке. Общий выход завершит программу, не продолжая загрузку по неверному адресу. При успехе выполнение идёт к следующей строке.

В полном примере после открытия также проверяются версия карты, наличие нужных функций и свободная память. Эти служебные проверки выполняет `check_card`. Для понимания загрузки звука достаточно знать их результат: либо можно продолжать, либо программа завершится с ошибкой.

2.2. Подготовить числа, которые станут звуком

Выберем простой формат **U8 mono**. U8 означает, что один отсчёт занимает один беззнаковый байт со значением 0..255. Середина этого диапазона - 128. Массив из одних значений 128 даёт тишину. Если значения периодически меняются выше и ниже середины, получается тон.

Наш массив содержит 32 значения 144, затем 32 значения 112. Это один период волны из 64 отсчётов. Повторим его четыре раза и получим 256 байт. `DUP`/`EDUP` в sjasmplus повторяют указанную группу при сборке; `DB` записывает байт в программу. Здесь ещё ничего не передаётся карте.

```
samples:
    DUP 4                   ; четыре одинаковых периода волны
        DUP 32
            DB 144          ; 32 отсчёта выше середины
        EDUP
        DUP 32
            DB 112          ; 32 отсчёта ниже середины
        EDUP
    EDUP
    ASSERT $-samples == TONE_BYTES ; 4 * (32 + 32) = 256 байт
```

Читать эти числа карта будет с частотой 8000 отсчётов в секунду. Один период занимает 64 отсчёта, поэтому основная частота гудка будет **8000 / 64 = 125 Гц**. Весь массив длится только 256 / 8000 = 0,032 секунды; чтобы услышать длинный гудок, позднее включим повтор массива до команды остановки.

Настройки примера вынесены в начало исходника. Их названия используются дальше вместо чисел, смысл которых пришлось бы вспоминать:

```
TONE_RATE   EQU 8000        ; 8000 отсчётов звука в секунду
TONE_BYTES  EQU 256         ; длина нашего массива в байтах
TONE_GAIN   EQU 16          ; громкость: 0 = тишина, 64 = максимум
PAN_CENTER  EQU 128         ; по центру между левым и правым выходами
PCM_U8      EQU 0           ; формат: один беззнаковый байт на отсчёт
```

2.3. Попросить карту выделить память

Команда **ALLOC** создаёт место для нового звука в памяти карты. Ей нужно передать вид данных и размер. Для этого подготовим небольшой блок аргументов в памяти Z80:

```
alloc_request:
    DB UG_KIND_RAW_PCM      ; будут обычные отсчёты, без заголовка файла
    DD TONE_BYTES           ; нужно 256 байт памяти на карте
```

`UG_KIND_RAW_PCM` означает массив отсчётов без файлового заголовка. `DD` записывает четырёхбайтовое число. Поэтому аргументы занимают **1 + 4 = 5 байт**. Сами 256 звуковых байт в эти пять байт не входят: их передадим следующим шагом.

Отправка команды выглядит так:

```
    ld a,UG_OP_ALLOC        ; команда: выделить память
    ld hl,alloc_request     ; адрес её аргументов в памяти Z80
    ld bc,UG_OP_ALLOC_REQ_LEN ; размер аргументов: 5 байт
    ld ix,UG_OP_ALLOC_RESP_LEN ; ожидаем ответ из 6 байт
    call rpc                ; отправить команду, получить response
    ret c                   ; ошибка: следующий шаг не выполнять
    ld hl,(response)        ; первые 2 байта: номер нового объекта
    ld (sound_handle),hl    ; запомнить этот номер
```

Здесь `rpc` - небольшая процедура нашего примера для отправки команды через библиотеку. У всех её вызовов один и тот же смысл параметров:

Регистр	Что передаём
A	Какую команду выполнить, например `UG_OP_ALLOC`
HL	Где в памяти Z80 лежат её аргументы
BC	Сколько байт занимают аргументы
IX	Сколько байт данных должно прийти в ответ

`UG_OP_ALLOC` и имена с окончанием `_LEN` определены в библиотечном файле `ug_wire.inc`. `_REQ_LEN` - размер аргументов, `_RESP_LEN` - размер ответа. За именем `UG_OP_ALLOC` стоит код команды 10h; в программе удобнее писать имя. Для ALLOC ответ состоит из двухбайтового номера объекта и четырёхбайтового фактического размера выделенной памяти: всего шесть байт.

`rpc` помещает ответ в массив `response`. Команда `LD HL,(response)` читает из его начала двухбайтовый **номер выделенного объекта**. Сохраняем номер в переменной `sound_handle`. Например, карта может вернуть 5; тогда дальнейшие операции с этим звуком используют 5. В другой раз номер может быть другим. Вручную подставлять постоянный номер нельзя.

2.4. Скопировать массив на карту

Место выделено, но звуковых данных в нём пока нет. Передадим массив `samples` из памяти Z80 в объект с полученным номером:

```
upload_sound:
    ld hl,0
    ld (UG_OFFSET),hl        ; смещение = 0: пишем с начала объекта
    ld (UG_OFFSET+2),hl      ; смещение занимает 4 байта, обнуляем оба слова
    ld hl,(sound_handle)     ; куда: объект в памяти карты
    ld de,samples            ; откуда: массив в памяти Z80
    ld bc,TONE_BYTES         ; сколько: 256 байт
    ld a,UG_TRANSPORT_BULK_VERIFIED ; передача с проверкой целостности
    call UG_WRITE
    ret
```

У `UG_WRITE` параметры отличаются от `rpc`: **HL выбирает объект на карте, DE указывает исходный массив Z80, BC задаёт количество байтов**. `UG_OFFSET` задаёт смещение внутри объекта. Сейчас оно равно нулю, поэтому запись начинается с первого байта. Смещение четырёхбайтовое; Z80 записывает его двумя двухбайтовыми командами.

`UG_TRANSPORT_BULK_VERIFIED` выбирает передачу с проверкой контрольной суммы. Библиотека сама договаривается с картой о передаче, отправляет блок и проверяет результат. При ошибке она вернёт C=1, и наш пример перейдёт к завершению. Писать свой цикл обращения к ISA-портам для этой задачи не требуется.

После успешного `UG_WRITE` копия массива находится на карте. Звук ещё не начался: карта пока не знает, с какой частотой и в каком формате его читать.

2.5. Сообщить карте, как читать массив

Одинаковые байты можно понимать как mono или stereo, как 8-битные или 16-битные значения, читать медленно или быстро. Команда **PCM_CONFIG** сообщает, как именно следует воспроизводить наш RAW-объект. В этом описании `DW` занимает два байта, `DD` - четыре, `DB` - один.

```
pcm_request:
    DW 0                    ; сюда allocate_sound запишет sound_handle
    DD TONE_RATE            ; частота: 8000 отсчётов в секунду
    DB 1                    ; один канал: mono
    DB 8                    ; 8 бит, то есть один байт на отсчёт
    DB PCM_U8               ; значения от 0 до 255, середина = 128
    DB 0                    ; внутренняя петля не используется
    DD 0                    ; отсчёты начинаются с первого байта объекта
    DD TONE_BYTES           ; длина массива в байтах
    DD 0                    ; начало внутренней петли: не используется
    DD 0                    ; конец внутренней петли: не используется
```

Первое `DW 0` - место под номер объекта. Процедура `allocate_sound` уже записала туда полученный `sound_handle`. Остальные поля описывают выбранный звук: один канал, один байт на отсчёт, частота 8000 Гц, длина 256 байт. Внутренняя петля позволила бы повторять часть массива; здесь она выключена.

```
describe_sound:
    ld a,UG_OP_PCM_CONFIG    ; команда: описать содержимое RAW
    ld hl,pcm_request        ; номер объекта, частота, формат, длина
    ld bc,UG_OP_PCM_CONFIG_REQ_LEN ; 26 байт аргументов
    ld ix,0                  ; данных в ответе нет, только успех/ошибка
    call rpc
    ret
```

SEAL означает «загрузка закончена, проверь объект». После этой команды карта разрешает использовать объект для воспроизведения. До SEAL он ещё считается загружаемым, даже если все байты уже переданы.

```
seal_request:
    DW 0                    ; сюда allocate_sound запишет sound_handle
    DB 0                    ; не задаём ожидаемую CRC всего объекта
    DD 0                    ; это поле тогда должно быть нулём

seal_sound:
    ld a,UG_OP_SEAL          ; команда: проверить и завершить загрузку
    ld hl,seal_request
    ld bc,UG_OP_SEAL_REQ_LEN ; 7 байт аргументов
    ld ix,UG_OP_SEAL_RESP_LEN ; 4 байта контрольной суммы в ответе
    call rpc
    ret
```

CRC - контрольная сумма для проверки целостности данных. В этом примере UG_WRITE уже проверил переданный блок. Поэтому дополнительное сравнение с заранее вычисленной CRC всего объекта выключено: оба поля после номера объекта нулевые. SEAL всё равно проверяет формат и возвращает вычисленную CRC. Порядок вызовов одинаков для своих звуков: загрузить данные, описать RAW, завершить загрузку, затем воспроизводить.

2.6. Включить звук

Теперь объект готов. Команде **PLAY_PCM** задаём его номер, количество повторов, громкость и положение между левым и правым выходами:

```
play_request:
    DW 0                    ; сюда allocate_sound запишет sound_handle
    DW 0                    ; повторять весь объект до команды STOP
    DB TONE_GAIN            ; небольшая громкость, 16 из 64
    DB PAN_CENTER           ; одинаковая громкость слева и справа
```

Первое слово уже заполнено номером объекта. Второе слово равно нулю: для этой команды **ноль означает повторять весь объект до STOP**. Значение 1 означало бы один проход, то есть для нашего массива всего 0,032 секунды. Pan=128 воспроизводит mono с одинаковой громкостью слева и справа; 0 перемещает звук влево, 255 - вправо.

```
start_sound:
    ld a,UG_OP_PLAY_PCM      ; команда: воспроизвести объект
    ld hl,play_request       ; номер объекта, повторы, громкость, панорама
    ld bc,UG_OP_PLAY_PCM_REQ_LEN ; 6 байт аргументов
    ld ix,UG_OP_PLAY_PCM_RESP_LEN ; 2 байта: номер запуска
    call rpc
    ret c
    ld hl,(response)
    ld (play_id),hl           ; нужен для остановки именно этого запуска
    xor a
    ret
```

После успешного вызова слышен низкий непрерывный тон. Карта повторяет массив сама: Z80 не должен снова и снова передавать те же 256 байт.

В ответ приходит **play_id - номер запуска**. Это другой номер, чем `sound_handle`. Номер объекта выбирает хранящиеся данные; номер запуска выбирает их текущее воспроизведение. Например, объект 5 может быть запущен с номером запуска 12. Останавливать нужно запуск 12, а удалять из памяти - объект 5. Оба номера в примере сохраняются из ответов карты.

2.7. Остановить звук и вернуть память

Остановка и удаление данных - два отдельных действия. Чтобы остановить наш тон, отправляем CONTROL с номером запуска и действием STOP:

```
stop_request:
    DW 0                     ; сюда stop_sound запишет play_id
    DB 0                     ; действие 0 = STOP (остановить)

stop_sound:
    ld hl,(play_id)           ; номер запуска, который требуется остановить
    ld (stop_request),hl
    ld a,UG_OP_CONTROL       ; управление воспроизведением
    ld hl,stop_request       ; номер запуска и действие STOP
    ld bc,UG_OP_CONTROL_REQ_LEN ; 3 байта аргументов
    ld ix,0                  ; ответ без дополнительных данных
    call rpc
    ret
```

После остановки объект остаётся на карте, и его можно было бы запустить снова. В нашем примере он больше не нужен, поэтому удаляем его командой FREE:

```
free_sound:
    ld a,UG_OP_FREE          ; команда: удалить объект из памяти карты
    ld hl,sound_handle       ; адрес переменной с номером объекта
    ld bc,UG_OP_FREE_REQ_LEN ; номер занимает 2 байта
    ld ix,0                  ; ответ без дополнительных данных
    call rpc
    ret
```

Здесь `LD HL,sound_handle` передаёт **адрес переменной** с номером объекта, потому что `rpc` читает аргументы по адресу HL. При загрузке в `UG_WRITE` использовалось `LD HL,(sound_handle)`: та процедура принимала сам номер. Скобки в этих двух вызовах имеют значение.

Воспроизведение останавливает звуковая часть карты. Она может ещё не успеть применить STOP к моменту первого FREE. Тогда придёт PINNED - «данные ещё используются». В полном примере процедура `cleanup` ограниченно ждёт и повторяет FREE. Она вызывается и при обычном выходе, и при ESC, и при ошибке. После очистки UG_CLOSE завершает работу с библиотекой. Сам UG_CLOSE не заменяет остановку звука и освобождение его памяти.

2.8. Дополнительная проверка: прочитать свои байты обратно

Чтобы убедиться, что в памяти карты лежит именно наш массив, пример также вызывает `verify_sound` перед воспроизведением. Эта проверка полезна при отладке загрузчика. Для самого звучания чтение обратно не требуется.

```
verify_sound:
    ld hl,(sound_handle)     ; читаем тот же объект
    ld de,readback           ; но в другой массив памяти Z80
    ld bc,TONE_BYTES
    ld a,UG_TRANSPORT_BULK_VERIFIED
    call UG_READ             ; UG_OFFSET остался равен нулю
    ret c
    ld hl,readback
    ld de,samples
    ld b,0                   ; цикл DJNZ при B=0 выполнит 256 сравнений
    call compare             ; проверить все прочитанные байты
    ret
```

`readback` - отдельный буфер на 256 байт в памяти Z80. Подпрограмма `compare` сравнивает его с исходным массивом `samples` и возвращает ошибку при отличии. Ноль в B использован из-за поведения команды DJNZ: после первого уменьшения он становится 255, поэтому весь цикл выполняется 256 раз.

2.9. Вся последовательность и первые изменения

Все разобранные шаги вызываются одной процедурой. `RET C` после каждого шага означает: если он не удался, дальнейшие действия со звуком не выполнять. Выход из `play_tone` всегда ведёт к общей очистке программы.

```
play_tone:
    call allocate_sound      ; получить место в памяти карты
    ret c
    call upload_sound        ; скопировать туда массив samples
    ret c
    call describe_sound      ; задать частоту и формат массива
    ret c
    call seal_sound          ; подтвердить: загрузка закончена
    ret c
    call verify_sound        ; прочитать обратно и сравнить
    ret c
    call start_sound         ; теперь включить воспроизведение
    ret c
    call wait_tone           ; подождать 1-2 секунды или ESC
    ret
```

`wait_tone` ждёт по часам DSS, проверяет ESC и наличие ошибки звука. Через 1-2 секунды или при ESC управление возвращается к общему выходу. Ожидания ограничены, поэтому неисправность карты или часов не должна повесить программу.

Попробуйте изменить один параметр и сравнить результат:

Изменение	Что должно произойти
`TONE_RATE = 16000` вместо 8000	Тот же массив читается вдвое быстрее: тон станет 250 Гц
`TONE_GAIN = 8` вместо 16	Тон станет тише
Последнее поле play_request = 0 вместо PAN_CENTER	Mono-тон будет слышен слева
Последнее поле play_request = 255	Mono-тон будет слышен справа

Частоту и громкость изменяйте в указанных пределах API: 8000..48000 Гц и 0..64. `TONE_BYTES` должен совпадать с действительным размером массива; проверка ASSERT в исходнике обнаружит несовпадение при сборке. Для дальнейшей работы с WAV или MOD сохраняется та же идея: получить номер объекта, загрузить данные, завершить загрузку, запустить и позднее освободить. Восстановление после сброса карты и потери ответа отдельно разобрано в 6.2.

3. Интерфейс со Sprinter

3.1. Адреса и ISA-окно

Карта декодирует A0-A9 ISA. BASE по умолчанию 02A0h, выровнен на 32 порта. Допустимый блок находится в 0200h..03FFh. Настраиваемый BULK alias также занимает 32 порта и не пересекается с BASE и другими функциями карты. Адреса соседних устройств программа автоматически не проверяет.

Sprinter отображает ISA I/O в C000h..FFFFh. Поэтому ISA-порт 02B3h доступен как ячейка C2B3h при открытом окне. Команды `OUT (C),A` к 02B3h сами по себе не заменяют этот доступ. PAGE3= D4h для слота 0, D6h для слота 1; SYSTEM=11h включает окно. Управление платформой выполняется через 00E2h (PAGE3), 1FFDh (SYSTEM), 9FBDh (старшие ISA-линии) и 003Ch (Turbo).

Код, стек, буферы и данные библиотеки должны лежать ниже C000h. DSS resident 0100h..3FFFh нельзя использовать как свободный буфер. UG_ISA_OPEN сохраняет PAGE3 и IFF, запрещает прерывания и выключает Turbo. UG_ISA_CLOSE возвращает PAGE3 и IFF, SYSTEM=01h и программное разрешение Turbo=3. Исходное состояние Turbo прочитать нельзя: порт доступен только для записи. Поэтому после закрытия устанавливается разрешение Turbo=3, а не сохранённое состояние. Аппаратное разрешение Turbo независимо от этого. Не вызывайте DSS, BIOS, дисковые операции или обработчики клавиатуры внутри открытого ISA-окна. Не делайте вложенных OPEN.

Библиотека открывает короткие окна на операции и закрывает их между ожиданиями. Обычные управляющие порции - до 8 байт; STREAM_FEED допускает 32 служебных байта. Прерывания не должны оставаться запрещёнными на всё время загрузки. Вариант STACK требует доказанной безопасности NMI; для учебных программ используется обычный FIXED transport. DI не блокирует NMI.

BULK alias - 32 адреса **одного FIFO**, не 32 байта PSRAM. После 32 LDI верните ISA-указатель к началу alias. Нельзя делать LDIR на 64 байта через соседние регистры. Без alias все байты передаются через один BASE+00h. GSALIAS - совершенно другая настройка: точные 00B3h/00BBh, те же GS-защёлки, что BASE+13h/1Bh. Она не переносит регистры расширенного API.

3.2. Командный и информационный флаги GS

STATUS = 7Eh | CB | DB: CB - бит 0, DB - бит 7. Проверяйте маски, а не равенство нулю. FFh может означать одновременно выставленные флаги; одного этого байта недостаточно, чтобы объявить карту отсутствующей.

Далее используются обозначения: SC - запись COMMAND; WC - ожидание CB=0; SD - запись DATA; WD - ожидание DB=0; GD - чтение DATA; WN - ожидание DB=1. Каждое ожидание ограничено тайм-аутом.

1. Если у команды есть pre, запишите один SD **до SC**. Не ждите WD между ними: карта потребит аргумент вместе с командой.
2. Выполните SC, затем WC.
3. Если есть post, передавайте каждый байт SD, WD по очереди.
4. Первый обязательный ответ читайте GD после WC; последующие - WN, GD. Необязательное старое значение читается непосредственно после WC без WN.

Входная и выходная защёлки разные. Новый SC отменяет недочитанный ответ и незавершённые post-аргументы. В LOAD действует другое правило: кроме D2/F3/F4, команды только квитируются и игнорируются. У post и legacy LOAD нет firmware idle timeout: паузы для диска допустимы, но host всё равно ограничивает WC/WD.

Низкоуровневая запись внутри уже открытого окна выглядит как `ld (0xC2B3),a` для SD или `ld (0xC2BB),a` для SC. Для прикладного кода используйте UG_GS_COMMAND: он выполняет последовательность и закрывает окна. Его дескриптор занимает 9 байт: command, has_pre, pre, post_count, post_ptr:u16, reply_count, reply_ptr:u16. Указатели little-endian.

3.3. Расширенные запросы и ответы

Запрос состоит из номера команды, длины и полезных данных. Полезные данные ниже называются payload; это аргументы команды без заголовка и CRC. Transfer - отдельная передача блока звуковых данных, которую начинают BEGIN и завершают END. Не путайте её с коротким управляющим запросом.

Все u16/u32/u64 передаются младшим байтом вперёд (little-endian), без выравнивания. Например, 1234h занимает два байта: 34h, 12h. Смещения в таблицах считаются от начала payload, размеры и смещения объекта - в байтах, если не указаны PCM frames. Один frame содержит значения всех каналов.

Перед запросом дождитесь X_STATUS.READY. В BASE+03h запишите opcode с битом 7 FRAME_CRC, длину payload (0..128), payload и при необходимости CRC32. Ответ читается из того же порта после REPLY: status, длина, payload, CRC32. CRC считается по двум байтам заголовка и payload; алгоритм CRC-32/ISO-HDLC, poly EDB88320h, init/xorout FFFFFFFFh, тест `123456789` -> CBF43926h. Ответ содержит CRC тогда и только тогда, когда запрос её потребовал. Одновременно допускается один запрос и один ответ. Полный максимальный frame с CRC - 134 байта. Звуковые данные идут отдельным блоком, не в payload команды.

READY не означает отсутствие transfer. Во время transfer разрешены INFO, MEM_INFO, STATS, SNAPSHOT, страницы FX/VOICE, LEGACY_INFO, OBJECT_INFO текущего объекта и END/CANCEL. Другие команды обычно возвращают BUSY. Чтение пустого native FIFO даёт FFh и UNDERFLOW, в отличие от сохраняющей последний байт GS-защёлки.

FRAME_CRC защищает команды; transport=80h/81h защищает данные блока. 00h=BULK FAST, 01h=GS FAST, 80h=BULK VERIFIED, 81h=GS VERIFIED. Проверка CRC управляющего кадра не означает проверку PCM-данных. Неверная длина кадра свыше 128 переводит транспорт в FAULT; нужен TRANSPORT_RESET, а не ожидание обычного ответа.

Тайм-ауты: простой RPC 1 с, BEGIN/END 2 с, SEAL 30 с, смена режима 10 с, MEM_TEST 120 с; брошенный grant карты - 5 с, неполный frame - 1 с. Библиотека с UG_DSS_TIMED использует DSS SYSTIME вне окна и конечный watchdog на случай остановившихся часов. Пример discover.asm без этого define использует ограниченные циклы опроса; firstpcm.asm включает UG_DSS_TIMED. Ни один вариант не должен зависать навсегда.

После потери ответа результат изменяющей команды может быть уже применён. TRANSPORT_RESET очищает обмен и отменяет неполный transfer, но не откатывает завершённые действия. Сверьте session, OBJECT_INFO/SNAPSHOT/STREAM_STATE; не повторяйте вслепую ALLOC, PLAY или STREAM_FEED. При смене session все сохранённые handles и instance IDs недействительны.

3.4. ABI библиотеки

UG_LIB_INIT вызывают один раз до UG_OPEN. UG_OPEN принимает A=слот, HL=BASE; он проверяет конфигурацию, но не выполняет MODE/RESET. UG_RPC: A=opcode, HL=payload, BC=длина, DE=буфер ответа, IX=ёмкость. На успехе CF=0, A=0, BC=фактическая длина; на ошибке CF=1, A=код. Всегда проверяйте ожидаемую длину ответа, не только CF. UG_LAST_SOURCE различает HOST=1, X=2, GS=3; UG_LAST_ERROR сохраняет код.

UG_WRITE/UG_READ: HL=handle, DE=буфер, BC=1..1024, A=transport, UG_OFFSET[4]=смещение. UG_FRAME_CRC принимает A=0/1. UG_SNAPSHOT возвращает 64 байта текущего контракта; безопасно выделить 128-байтовый буфер, как делают существующие клиенты. UG_CLOSE отменяет собственный незавершённый transfer; он не освобождает ваши объекты и не останавливает звук автоматически. IX/IY сохраняются библиотекой, AF/BC/DE/HL и альтернативные регистры могут меняться. Вызовы сериализованы; библиотека не рекурсивна и не вызывается из ISR.

4. Система команд GS 1.x

Номера GS шестнадцатеричные и относятся к GS COMMAND, не к opcode native API. Например GS 32h приостанавливает MOD, а native 32h принимает play_id и action. Таблица ниже содержит все поддержанные команды, их аргументы и результат. `[old]/[module]/[fx]` - необязательная OUT-защёлка после WC, без WN. Номера и привычные названия сохранены; технические символы GS_* из схемы приведены для поиска в исходниках. В примерах обмена pre и post означают значения из соседней колонки, ×N - повтор N раз. Это обозначения протокола, не ASM-листинг; для групп direct FX показан первый канал группы.

GS hex	pre; post	out	Назначение и влияние
00	-	-	GS_RESET_FLAGS. Вне LOAD: сбросить CB/DB и незавершённые legacy аргументы; готовые объекты и музыку сохранить. Пример обмена: SC 00h, WC.
08	-	-	GS_NOP. No-op, квитировать CB; не отменять LOAD, не очищать GS_ERROR. Пример обмена: SC 08h, WC.
20	-	3 LE	GS_POOL_SIZE. Объём пула PSRAM в байтах. Пример обмена: SC 20h, WC, GD; WN/GD × 2.
21	-	3 LE	GS_POOL_FREE. Свободные байты пула; это 21h, несмотря на опечатку 20h в старом руководстве. Пример обмена: SC 21h, WC, GD; WN/GD × 2.
23	-	1	GS_POOL_PAGES. `min(pool_bytes/32768,255)`; насыщение необходимо для 8 МиБ. Пример обмена: SC 23h, WC, GD.
2A	volume; -	[old]	GS_MUSIC_MASTER. Music master 0…64. Пример обмена: SD(pre), SC 2Ah, WC, GD(out), необязательно.
2B	volume; -	[old]	GS_FX_MASTER. FX master 0…64. Пример обмена: SD(pre), SC 2Bh, WC, GD(out), необязательно.
2E	fx; -	[old]	GS_SELECT_FX. Выбрать текущий legacy FX; 0 выбирает последний загруженный. Пример обмена: SD(pre), SC 2Eh, WC, GD(out), необязательно.
30	-	[module]	GS_LOAD_MODULE. Начать загрузку модуля, см.. Пример обмена: SC 30h, WC, SD/WD для каждого байта файла, SC D2h, WC, GD(handle).
31	module; -	[module]	GS_PLAY_MODULE. Играть модуль с начала: speed=6, BPM=125, music gain=64; master сохранить. Пример обмена: SD(pre), SC 31h, WC, GD(out), необязательно.
32	-	[module]	GS_PAUSE_MODULE. Приостановить текущий MOD, сохранить позицию; FX продолжаются; 0 без MOD. Пример обмена: SC 32h, WC, GD(out), необязательно.
33	-	[module]	GS_RESUME_MODULE. Продолжить MOD; без MOD - no-op с выходом 0, при уже играющем MOD - идемпотентно. Пример обмена: SC 33h, WC, GD(out), необязательно.
35	volume; -	[old]	GS_MUSIC_GAIN. Music gain 0…64, независимо от master. Пример обмена: SD(pre), SC 35h, WC, GD(out), необязательно.
36	-	[FFh]	GS_OUT_FF. Записать FFh в выходную защёлку, без звукового эффекта. Пример обмена: SC 36h, WC, GD.
37	-	-	GS_FREE_LEGACY. Освободить все legacy-owned MOD/FX и сбросить их выбор; независимые EXT объекты сохранить. Пример обмена: SC 37h, WC.
38	-	[fx]	GS_LOAD_FX. Начать загрузку FX U8 mono. Пример обмена: SC 38h, WC, SD/WD для каждого байта файла, SC D2h, WC, GD(handle).
39	fx; -	-	GS_PLAY_FX_SEEK. Запустить FX по seek masks и priority. Пример обмена: SD(pre), SC 39h, WC.

GS hex	pre; post	out	Назначение и влияние
3A	mask; -	-	GS_STOP_FX_MASK. Остановить legacy FX: bit7→GS0, bit6→GS1, bit5→GS2, bit4→GS3; FFh останавливает все legacy FX, не EXT dedicated lanes. Пример обмена: SD(pre), SC 3Ah, WC.
3D	volume; -	[old]	GS_FX_GAIN. FX gain 0…64. Пример обмена: SD(pre), SC 3Dh, WC, GD(out), необязательно.
3E	sign; -	[fx]	GS_LOAD_FX_SIGNED. Загрузка FX: **0=S8, 1=U8** по ROM; прочее ошибка. Пример обмена: SD(pre), SC 3Eh, WC, SD/WD для каждого байта файла, SC D2h, WC, GD(handle).
40	note; -	[old]	GS_FX_NOTE. Нота текущего FX, 0…95. Пример обмена: SD(pre), SC 40h, WC, GD(out), необязательно.
41	volume; -	[old]	GS_FX_VOLUME. Громкость текущего FX, 0…64. Пример обмена: SD(pre), SC 41h, WC, GD(out), необязательно.
42	fine; -	[old]	GS_FX_FINETUNE. Finetune текущего FX, правило ниже. Пример обмена: SD(pre), SC 42h, WC, GD(out), необязательно.
45	priority; -	[old]	GS_FX_PRIORITY. Приоритет текущего FX, 1…254. Пример обмена: SD(pre), SC 45h, WC, GD(out), необязательно.
46	mask; -	[old]	GS_SEEK_FIRST. SeekFirst, младшие четыре бита в GS-нумерации. Пример обмена: SD(pre), SC 46h, WC, GD(out), необязательно.
47	mask; -	[old]	GS_SEEK_LAST. SeekLast, младшие четыре бита в GS-нумерации. Пример обмена: SD(pre), SC 47h, WC, GD(out), необязательно.
48	loop_start.lo; mid,hi	-	GS_LOOP_START. Начало петли U8/S8 в байтах; hi=FFh отключает петлю. Пример обмена: SD(pre), SC 48h, WC, SD/WD × 2.
49	loop_end.lo; mid,hi	-	GS_LOOP_END. Исключительная правая граница петли в байтах. Пример обмена: SD(pre), SC 49h, WC, SD/WD × 2.
60	-	1	GS_ORDER. Текущий order; 0 без MOD. Пример обмена: SC 60h, WC, GD.
61	-	1	GS_ROW. Текущая row; 0 без MOD. Пример обмена: SC 61h, WC, GD.
62	-	1	GS_ORDER_ROW. `((order & 3)<<6) | row`; 0 без MOD. Пример обмена: SC 62h, WC, GD.
63	-	4	GS_NOTES. Ноты MOD 0,1,2,3 в порядке CHANS и флаги изменения, не DAC-порядок. Пример обмена: SC 63h, WC, GD; WN/GD × 3.
64	-	4	GS_VOLUMES. Громкости MOD 0,1,2,3 в порядке CHANS и флаги изменения. Пример обмена: SC 64h, WC, GD; WN/GD × 3.
65	order; -	-	GS_SEEK_ORDER. Seek к order=N, row=0; сбросить канальные эффекты/speed/BPM как SEEK_MOD. Пример обмена: SD(pre), SC 65h, WC.
66	value; -	-	GS_SPEED_BPM. 1…31=speed; 32…255=BPM; 0=F00 с учётом player_mode. Пример обмена: SD(pre), SC 66h, WC.
67	-	1	GS_SPEED. Текущий speed. Пример обмена: SC 67h, WC, GD.
68	-	1	GS_BPM. Текущий BPM. Пример обмена: SC 68h, WC, GD.
69	-	-	GS_EXTRA_TICK. Один дополнительный MOD tick при любом BUSY; утверждённая native-семантика. Пример обмена: SC 69h, WC.
6A	mode; -	[old]	GS_F00_MODE. 0: F00 останавливает; ненулевое: игнорировать F00. Пример обмена: SD(pre), SC 6Ah, WC, GD(out), необязательно.
6B	необязательный lo; hi	-	GS_RELOOPER. Настройка relooper в словах. Пример обмена: SD(pre), SC 6Bh, WC, SD/WD × 1.

GS hex	pre; post	out	Назначение и влияние
80…83	fx; -	-	GS_FX_CH0. Принудительно запустить FX в GS-канале c=opcode−80h. Пример обмена: SD(pre), SC 80h, WC.
88…8B	fx; note	-	GS_FX_CH_NOTE. То же, с нотой только для этого запуска. Пример обмена: SD(pre), SC 88h, WC, SD/WD × 1.
90…93	fx; volume	-	GS_FX_CH_VOL. То же, с громкостью только для этого запуска. Пример обмена: SD(pre), SC 90h, WC, SD/WD × 1.
98…9B	fx; note,volume	-	GS_FX_CH_NV. То же, с нотой и громкостью только для этого запуска. Пример обмена: SD(pre), SC 98h, WC, SD/WD × 2.
A0…A3	note; -	-	GS_SET_NOTE. Изменить ноту активного FX в GS-канале. Пример обмена: SD(pre), SC A0h, WC.
A8…AB	volume; -	-	GS_SET_VOL. Изменить громкость активного FX в GS-канале. Пример обмена: SD(pre), SC A8h, WC.
D0	-	-	GS_EXT_D0. NORMAL: ack/no-op; POST: прервать ожидание, затем no-op; LOAD: ack+ignore. Не API ошибки. Пример обмена: SC D0h, WC.
D1	-	-	GS_EXT_D1. LOAD: идемпотентный маркер, ack+ignore; NORMAL: no-op; POST: прервать ожидание, затем no-op. Пример обмена: SC D1h, WC.
D2	-	[handle]	GS_LOAD_CLOSE. Закрыть stream, проверить/опубликовать объект; 0 при ошибке. Пример обмена: SC D2h, WC, GD.
F0	-	1	GS_READ_ERROR. Прочитать GS_ERROR, не очищать. Пример обмена: SC F0h, WC, GD.
F3	-	-	GS_WARM_RESET. Warm reset всего хранилища, сохранить 6A/6B; сменить session. Пример обмена: SC F3h, WC.
F4	-	-	GS_COLD_RESET. Cold reset хранилища + quick memory test; 6A=0,6B=0; сменить session. Пример обмена: SC F4h, WC.
F5	-	-	GS_BUSY_ON. Busy On: приостановить автоматическое обслуживание MOD, не немедленный mute;. Пример обмена: SC F5h, WC.
F6	-	-	GS_BUSY_OFF. Busy Off: вернуть автоматическое обслуживание MOD, без накопленных пропущенных тиков. Пример обмена: SC F6h, WC.

4.1. Как применять таблицу

Примеры ниже - **обозначения обмена**, не самостоятельные ASM-программы. Для любой строки порядок pre/SC/WC/post/GD определён в 3.2. Например:

```
Объём:      SC 20; WC; GD low; WN; GD mid; WN; GD high
Music gain: SD 20h; SC 35h; WC; GD old (необязательно)
Петля:      SD 00h; SC 48h; WC; SD 01h; WD; SD 00h; WD
            SD 00h; SC 49h; WC; SD 02h; WD; SD 00h; WD
FX lane 1:  SD fx; SC 99h; WC; SD 60; WD; SD 16; WD
Пауза:      SC 32h; WC
Продолжить: SC 33h; WC
```

Петля в этом примере охватывает байты 256..511, loop_end=512 исключительно. 99h выбирает GS lane 1, note=60 и volume=16 только для данного запуска. Управляющие команды не возвращают native status: после отказа читайте F0h или GS_ERROR. Ошибка sticky; успешная команда не обязана её стереть.

4.2. Загрузка MOD и FX

Для MOD выполните 30h/WC, при необходимости прочитайте предварительный номер, передайте исходный файл SD/WD побайтно, затем D2h/WC/GD. Ненулевой результат D2 означает публикацию; 0 означает ошибку, которую нужно прочесть. Запуск: pre=1, SC31/WC. Для U8 FX используется 38h, для S8 - pre=0, SC3E. D1 - допустимый идемпотентный маркер, но не обязательный заголовок данных. Предварительный handle ещё не означает READY. D2 проверяет размер и формат.

Legacy MOD имеет байтовый номер 1, FX - 1..60 в отдельном пространстве. Эти номера нельзя передавать в native FREE/PLAY. LEGACY_INFO возвращает соответствующие **native u16 handles**. Значение 0 в аргументе выбора обычно обозначает текущий объект; у 2E - последний загруженный FX. 30h может заменить прежний legacy MOD: сначала убедитесь, что управляете им. В учебном GS-сценарии требуется пустое хранилище, иначе возвращается BUSY.

В LOAD 00h и 08h не отменяют загрузку. D0/D1 также только подтверждаются. Для завершения используйте D2; F3/F4 сбрасывают всё хранилище и не подходят для обычной очистки приложения. На ошибке завершают собственный LOAD, сверяют session и принадлежность, затем удаляют свои объекты.

4.3. Отличия, которые влияют на программу

08h - no-op с квитированием, **не** синоним Reset flags 00h старого текста. Свободная память читается 21h; повтор 20h в старом описании - опечатка. 3Eh: sign=0 означает S8, sign=1 означает U8; противоположное утверждение в историческом руководстве не соответствует ROM и UltraGS. 23h насыщается на 255 страниц по 32768 байт; для полного объёма используйте 20h (для 8 МиБ ответ 00 00 80) или MEM_INFO.

32h - пауза с сохранением позиции, а не освобождение MOD. FX продолжаются. 33h продолжает; без MOD результат 0, повтор при играющем MOD идемпотентен. 31h запускает с начала, speed=6, BPM=125, music gain=64; master сохраняется. 65h сбрасывает эффекты каналов и темп на 6/125, выбирая order и row=0.

GS0..GS3 для direct FX и масок соответствуют MOD-каналам 0,3,1,2. Но ответы 63h/64h идут в естественном порядке MOD 0,1,2,3. Их bit7=1, если значение не менялось с предыдущего чтения этой команды. Low7 - нота 0..95 (127=нет ноты) либо громкость 0..64. Первый ответ после reset: 7F для нот, 00 для громкостей; повтор - FF/80. SNAPSHOT не сбрасывает эти флаги. Для MOD8/32 исторические 63/64 всё равно показывают только первые четыре канала.

39h ищет свободный младший GS-канал в SeekFirst, затем SeekLast; затем может вытеснить минимальный priority из SeekLast, только если новый priority строго больше. Direct 80..9B заменяет указанный legacy выход независимо от priority. При foreground PCM legacy FX запуск запрещён (STATE), чтобы не заменять половину stereo. При MOD скрытое состояние канала продолжает рассчитываться.

FX defaults: note=60, volume=64, fine=0, priority=128, обе seek masks=0Fh, петля выключена. 40..49 меняют defaults объекта, A0/A8 - текущий экземпляр. Нота 36 соответствует period=856, 48 - 428, 60 - 214; скорость PAL 3546895/period Гц. Finetune сохраняет байт, индекс `(fine>>1)&15`, 8..15 означают -8..-1 с шагом 1/8 полутона. Legacy volume выше 64 насыщается; native API возвращает BAD_ARGUMENT. При петле должно быть start<end<=size; hi=FF у 48h выключает петлю, промежуточная незавершённая пара допустима до PLAY.

6Ah: 0 включает остановку по F00, ненулевое значение игнорирует F00. На UltraGS остановка F00 сохраняет позицию и ресурсы как пауза. GS66 с нулём также использует эту политику. F5 запрещает автоматические MOD ticks, но микшер продолжает текущие ноты/петли и независимые FX; это не mute. 69 ставит один дополнительный tick, F6 возобновляет таймер без catch-up. Это нормализованная семантика UltraGS, не точная эмуляция ISR ROM. 6B принимает 0..16384 слов; короткая форма или неверное число дают 512, 0 выключает. Данные и объём памяти не меняются: физическое размножение петель на нативном движке не требуется.

37h удаляет legacy-owned MOD и FX, сохраняя независимые native объекты. F3/F4 удаляют **все** объекты и меняют session; F3 сохраняет 6A/6B, F4 сбрасывает их и выполняет quick memory test. Это не reboot multifw, BASE и runtime GSALIAS сохраняются. Не используйте F3/F4 как cleanup.

4.4. Остальные исторические команды

Команды	Историческое назначение	UltraGS в NORMAL
01..07, 09..0F	Прямой DAC/volume, Covox/SounDrive/Ultravox	GS_ERROR=01h, без исполнения
10..1F	Порты, CALL/JUMP, загрузка RAM, POKE/PEEK, регистр DE	GS_ERROR=01h
22	Внутренняя переменная GS	GS_ERROR=01h
24..29, 2C..2D, 2F, 34, 3B..3C, 3F	Резерв/неопределённые операции	GS_ERROR=01h
43..44, 4A..5F, 6C..7F	Резерв вне музыкального профиля	GS_ERROR=01h
Промежутки между direct FX группами, AC..CF, D3..EF	Резерв	GS_ERROR=01h
F1..F2, F7..FF	Резерв, HX, тестовые режимы, включая FA	GS_ERROR=01h

В POST новый SC прерывает ожидание и затем обрабатывается как команда. В LOAD приоритет имеет правило ack+ignore, а не таблица NORMAL. Неизвестная команда не запускает встроенный Z80 и не оставляет CB навечно.

5. Расширенный UltraGS API 1.0

5.1. Обнаружение и память

INFO принимает только пустой payload; в текущей реализации лишний байт возвращает BAD_OPCODE (не LENGTH). INFO доступен и при неисправной памяти. boot_state: 0=ready, 1=PSRAM failure, 2=startup test, 3=audio failure. fw_version=(major<<16)|(minor<<8)|patch. Session - u64, не время и не адрес. До проверенного INFO40/VLS1 не выполняйте ALLOC/PLAY. CAPS возвращает 44 байта; music_limit и fx_limit относятся к **одному** пулу и не складываются. Форматы проверяются отдельно от features.

MEM_INFO различает physical, pool, free и largest. Для одного ALLOC важен largest, не только суммарный free. ALLOC создаёт UPLOADING, округляет резерв до 16 байт; kind=0 BLOB, 1 RAW, 2 WAV, 3 MOD. Kind4 создают STREAM_CREATE. В session не более 128 живых объектов; handle не переиспользуется до reset, 65535 выдач исчерпывают пространство даже при освобождении прежних объектов. OBJECT_ENUM(after=0) начинает обход, next=0 завершает. OBJECT_INFO сообщает state, pins, размер, uploaded, CRC и разобранные параметры звука.

FREE удаляет объект только без pins, иначе PINNED. Pin означает, что объект ещё используется: проигрывается, загружается или читается. SEAL разрешён после полной загрузки; flags=0 требует expected_crc=0, flags=1 сравнивает CRC всего файла. Успех переводит в READY; данные READY неизменяемы. Ошибка FORMAT/CRC не позволяет запускать объект. Читайте OBJECT_INFO, затем освобождайте свой объект. RAW требует PCM_CONFIG до SEAL, WAV/MOD разбираются на карте.

MEM_TEST mode=0 quick, 1 full требует пустой store и отсутствие звука/transfer. На успехе first_bad=FFFFFFFFh. При PSRAM payload с результатом теста сохраняется: не отбрасывайте tested, first_bad, expected/actual. RESET_STORE с ключом 55475253h и level=0/1 - явное глобальное обслуживание; не часть загрузки/cleanup.

5.2. Загрузка и чтение

Grant - разрешение карты передать один блок. После BEGIN нельзя сразу писать или читать данные: сначала дождитесь соответствующего флага GRANT. Процедуры UG_WRITE и UG_READ выполняют эти ожидания внутри библиотеки.

WRITE_BEGIN задаёт handle, offset, N=1..1024 и transport. После ответа с token дождитесь WRITE_GRANT и передайте ровно N байт. WRITE_END(token) подтверждает commit. Только успешный END увеличивает подтверждённый uploaded. Загрузка последовательная; BLOB дополнительно допускает перезапись уже загруженного диапазона. Одновременно возможен только один transfer.

READ_BEGIN задаёт тот же диапазон в пределах uploaded. VERIFIED возвращает token и CRC; дождитесь READ_GRANT, прочитайте ровно N байт, передайте READ_END(token,host_crc). FAST не содержит CRC. TRANSFER_CANCEL принимает token, а X_CONTROL A1 отменяет текущий transfer без payload. Ранее завершённые блоки сохраняются. Чтение READY разрешено во время звучания; запись в него нет.

У PCM_CONFIG descriptor 24 байта. Rate=8000..48000, channels=1/2, bits=8/16, format=0 U8, 1 S8, 2 S16LE. Для 16 бит только format2. RAW data_offset=0, data_length=size, кратно размеру frame. flags bit0 включает forward loop; тогда loop_start<loop_end<=frames, в единицах **frames**, не байтов. Без петли оба поля нулевые. Для WAV PCM_CONFIG запрещён. RIFF PCM WAV допускает неизвестные chunks и padding нечётных размеров, но ровно один fmt и data. Дубликаты, compressed/extensible/float, RIFX/RF64, неверные block_align/byte_rate или усечённые chunks дают FORMAT.

5.3. MOD, PCM и FX

У карты один основной источник звука (foreground): MOD, PCM или поток. Независимые эффекты FX играют вместе с ним. Instance - отдельный запуск объекта; PLAY возвращает ID этого запуска. Физические голоса - ресурсы, которые нужны для его воспроизведения.

PLAY_PCM/PLAY_MOD заменяют один foreground после полной проверки нового запуска. Ошибка не должна останавливать прежнюю музыку. PLAY_PCM принимает repeats=1 для одного прохода, 0 для бесконечного; встроенная RAW-петля требует repeats=1. Индивидуальная громкость gain и общие уровни микшера задаются в диапазоне 0..64; панорама pan=0..255: 0 лево, 128 центр, 255 право. Для stereo pan работает как баланс: в центре оба канала без ослабления. Для mono он линейно распределяет звук между левым и правым выходами. Перед панорамированием MOD с 5..8, 9..16 и 17..32 каналами ослабляется соответственно в 2, 4 и 8 раз, чтобы оставить запас при суммировании. У MOD с 1..4 каналами и у PCM/FX такого ослабления нет. Сумма громких FX всё равно может превысить диапазон signed16; это слышно как искажение и учитывается счётчиком clip_samples.

PLAY_MOD flags: bit0 LOOP, bit1 IGNORE_F00, bit2 PAUSE_F00. Последние два требуют CAPS bit12 и взаимоисключающие. Если оба равны 0, политика следует GS6A, включая последующие изменения. Явная политика сохраняется при PAUSE/RESUME/SEEK/LOOP и не меняется от GS6A.

CONTROL: action 0 STOP, 1 PAUSE, 2 RESUME. Play_id=0 выбирает текущий foreground, но для безопасной очистки передавайте свой точный ID. STOP снимает pins, но не освобождает объект. PAUSE сохраняет голоса и фазу; FX продолжаются. EOF нельзя продолжить RESUME: нужен новый PLAY. SEEK_MOD принимает допустимый order и row=0..63, сбрасывает эффекты и tempo на 6/125, сохраняет паузу. SET_TEMPO: speed=1..31, BPM=32..255. SET_PAN принимает естественный логический канал MOD (u16), не физический voice_id. В текущем диспетчере допускаются только каналы 0..7 даже у MOD с 9..32 каналами (иначе RANGE); настройка живёт до PLAY/SEEK или команды панорамы в самом MOD.

PLAY_FX создаёт независимый instance. preferred_voice=FFFFh - автоматический выбор; иначе номер первого физического голоса. Для stereo второй голос выбирается отдельно, соседство и чётность не требуются. Priority=1..254; flags bit0 ALLOW_FX_STEAL, bit1 LOOP_OBJECT. LOOP_OBJECT вместе с внутренней RAW-петлёй запрещён. Steal может целиком вытеснить только FX меньшего priority, не музыку. При недостатке голосов NO_VOICE атомарен: половина stereo не стартует.

STOP_FX принимает **instance_id**, не маску; 0 останавливает все native FX и не подходит для cleanup собственного эффекта среди чужих. CONTROL_FX принимает точный ненулевой ID и action=0/1/2. GS legacy masks не являются масками native голосов. Pending PLAY уже резервирует ресурсы до render-границы. PLAY/STOP подтверждают принятие события; освобождение может потребовать дождаться применения, затем повторить FREE при PINNED с ограниченным сроком.

5.4. Streaming

Один PCM_STREAM создаёт кольцо в PSRAM. Размер BUFFER, PREFILL и LOW в API передаются в **байтах**, кратных channels*bits/8: BUFFER>0, 0<PREFILL<=BUFFER, 0<=LOW<PREFILL. Степень двойки не нужна. Второй STREAM_CREATE даёт BUSY; ALLOC kind4 не заменяет CREATE.

STREAM_FEED - специальная операция: отправьте request с sequence (первый=1), N=1..1024 **байт** и transport=00/80. N должен быть кратен размеру PCM-кадра (`channels * bits / 8`). Например, 1024 байта S16 stereo - это 256 кадров. Ждите WRITE_GRANT либо REPLY с ошибкой. Передайте N байт через BULK, затем ждите единственный ответ next_sequence и published. **WRITE_END здесь не нужен.** Опубликованные и потреблённые счётчики измеряются в frames (u64), fill/free - в байтах. Обычный UG_RPC не выполняет промежуточную передачу данных. В приложении UGPLAY он собран с APP_PLAY и вызывает STREAM_BULK при WRITE_GRANT. Этот вариант вместе с ugstream.asm разобран в разделе 8.5.

В REQ находятся параметры STREAM_FEED: 9 байт, либо 13 байт с CRC данных. UG_RPC копирует их в служебный буфер команды. Звуковой блок (до 1024 байт) STREAM_BULK передаёт непосредственно из файлового буфера Z80 через BULK после WRITE_GRANT.

START занимает foreground и 1/2 голоса, оставляя независимые FX. До PREFILL карта выдаёт тишину. При нехватке данных наступает buffering с сохранением позиции, один underrun на эпизод; после PREFILL звучание продолжается. EOF доигрывает остаток даже меньше PREFILL. После EOF объект сохраняется для STATE до CLOSE. CLOSE/STOP/замещение освобождают кольцо на render-границе. Для собственного потока используйте CLOSE(handle), не глобальный STOP.

STREAM_STATE state: 0 created, 1 playing, 2 paused, 3 EOF, 4 error, 5 buffering. Flags: bit0 EOF принят, bit1 требуется пополнение, bit2 закрытие принято. Consumed может опережать слышимую position на held interpolation taps; fraction - Q0.32 дробная часть. При потере ACK сделайте TRANSPORT_RESET, прочитайте STATE и сравните next_sequence. Повтор вслепую дублирует данные.

Практическое ограничение: непрерывный дисковый 44100/S16LE/stereo streaming на Sprinter пока не обеспечен. Подтверждены слышимые паузы и underruns даже при RESULT PASS/EOF. Такой поток требует 176400 байт/с без учёта протокола. Большой BUFFER сглаживает паузы, но не исправляет недостаточную среднюю скорость. UGPF ускоряет только BULK writes; это не гарантия непрерывности. Для обучения используйте короткий mono U8 WAV 8000 Гц из примера, проверяйте underruns и звук. В своём цикле подачи сравнивайте STREAM_STATE.underruns до и после воспроизведения. Успешные ответы FEED и достижение EOF сами по себе не подтверждают, что звук шёл без пауз.

5.5. Снимки, статистика и ошибки

SNAPSHOT фиксирует 64-байтовую сводку. Для VOICE_STATUS/FX_STATUS передайте session и snapshot_seq именно из неё, first=0 и max_records (до 5 voices или 6 FX за страницу). Ответ содержит PAGE_HEADER и записи. Продвигайте first на count; first==total даёт пустой конец. Новый SNAPSHOT/SNAP_LATCH заменяет удерживаемую копию. При STATE начните обход заново с нового снимка. Не смешивайте страницы разных ключей. VOICE_STATUS содержит весь пул с idle записями; FX_STATUS - экземпляры, stereo представлен одной записью FXREC с channels=2. Поле count в заголовке страницы считает записи, а не голоса.

SNAPSHOT.state: 0 stopped, 1 playing, 2 paused, 3 EOF, 4 error. Kind: 0 none, 1 PCM, 2 MOD, 4 stream (3 зарезервирован для очереди). Kind снимка не равен object kind: WAV-объект kind2 играет как PCM kind1. Source_frame у MOD в summary равен 0: позицию песни задают order/row/tick, а фазу каждого sample - VOICEREC.source_frame. Output_frames - u64 счётчик выходных кадров 48000 Гц; source_frame - исходные кадры (u32).

VOICEREC.flags: bit0 active, bit1 FX, bit2 paused, bit3 reserved (голос занят). Logical_channel=FFFFh у FX/idle; note=127 означает отсутствие MOD-ноты. Volume для MOD - 0..64 из трекера, для PCM - gain; period значим для MOD. Peak - абсолютная амплитуда в линейных единицах signed16, не dB. FXREC.state=1 playing или 2 paused; flags bit0 legacy override, bit1 повтор всего объекта. Idle голоса остаются в VOICE_STATUS; idle FX-записей в FX_STATUS нет. Transfer_done/length - байты текущего блока, не размер файла.

STATS возвращает 16 u32 счётчиков, включая underruns, protocol/PSRAM errors, максимальное время render и clips. Сравнивайте дельты; CLEAR_STATS глобален и может мешать другому наблюдателю. Пауза не освобождает голоса: voice_count считает резерв, а не только звучащие в эту секунду каналы. X_CONTROL принимает одно значение: A0 TRANSPORT_RESET, A1 TRANSFER_CANCEL, A2 CLEAR_ERRORS, A3 SNAP_LATCH. Это не битовая маска.

Размеры всех запросов и ответов, смещения полей и вложенные структуры приведены в приложениях 9.7-9.8. При реализации собственной библиотеки используйте их вместе с правилами обмена из главы 3.

6. Память, запуски и завершение программы

6.1. Три разных номера: объект, запуск, голос

У загруженного звука есть номер объекта - handle. Отдельное воспроизведение этого звука имеет свой номер запуска - instance_id или play_id. Для получения звука карта назначает запуску физические голоса. Программа обычно выбирает объект, а подбор свободных голосов поручает карте.

Например, один stereo WAV с handle=17 можно запустить двумя эффектами и получить номера запусков 5 и 6. Для данных требуется один объект; для двух stereo-запусков - четыре голоса. STOP_FX(5) остановит только первый эффект. FREE(17) пока вернёт PINNED: второй запуск ещё использует эти данные. После STOP_FX(6) и завершения остановки объект можно удалить. Числа в этом примере условные: всегда сохраняйте номера из ответов карты.

Восьмиканальный MOD занимает восемь из 32 голосов. Оставшихся 24 хватит, например, на 24 mono или 12 stereo FX, если других потребителей нет. MOD с 32 каналами занимает все голоса. Пауза не освобождает голоса: при продолжении должны сохраниться состояние и положение звука.

6.2. Зачем существует session и когда о ней нужно знать

`session` - служебный идентификатор текущего состояния хранилища карты. Карта меняет его при сбросе хранилища или перезапуске. Это не учётная запись пользователя и не номер звука. Пользователь его не выбирает, в настройки не вводит и команде PLAY не передаёт.

Он нужен программе, которая хранит номера объектов и должна безопасно пережить сбой. Предположим, программа загрузила звук и получила handle=5. Затем карту сбросили: звук исчез. Позднее номер 5 может быть присвоен другому объекту. Если продолжить пользоваться сохранённой пятёркой, можно обратиться к чужим данным. Изменившаяся session позволяет обнаружить это до FREE или PLAY.

Порядок защиты простой: после открытия сохранить поле session из INFO; при восстановлении обмена прочитать INFO снова и сравнить восемь байт. Если значение изменилось, забыть старые номера в переменных программы. Посылать FREE для этих номеров в новое хранилище уже нельзя.

Вот почему в служебной части первого примера есть копирование:

```
    ld hl,response+12        ; поле session в ответе INFO
    ld de,saved_session      ; место для его копии в памяти Z80
    ld bc,8                  ; идентификатор занимает восемь байт
    ldir                     ; сохранить, ничего не посылая карте
```

`response+12` - начало поля session в ответе INFO, согласно таблице 9.7. `saved_session` - имя буфера в памяти Z80. Строка `LD DE,saved_session` всего лишь выбирает, куда скопировать байты командой LDIR. Она не начинает новый сеанс на карте и не включает воспроизведение. Это проверка для восстановления после ошибки; само получение звука объясняется без неё. Полная программа сохраняет эту защиту, чтобы её можно было безопасно запускать.

Есть и другая неисправность: команда выполнена, но её ответ не дошёл. Например, PLAY уже включил звук, а программа не получила play_id. Повтор PLAY вслепую может запустить звук заново. Поэтому программа читает SNAPSHOT, находит запуск своего объекта и получает его номер оттуда. Аналогично после потери ответа STREAM_FEED сначала проверяется состояние потока, чтобы не послать один блок дважды.

Первый PCM-пример заранее требует пустого хранилища и единственного клиента. Поэтому после потери ответа ALLOC он может найти свой единственный объект через OBJECT_ENUM. Этот приём нельзя переносить в программу, которая работает среди чужих объектов: ей нужна собственная учётная информация о загрузках.

6.3. Что освобождать при выходе

Обычный путь объекта: выделить память, загрузить байты, для RAW задать PCM_CONFIG, выполнить SEAL, запустить звук, остановить его, вызвать FREE. BLOB хранит произвольные данные и читается обратно, но не воспроизводится. MOD и WAV описывают свой формат сами, поэтому PCM_CONFIG им не нужен.

Программа сохраняет номера всех своих объектов и запусков. При выходе она завершает или отменяет свою передачу, останавливает свои запуски, дожидается освобождения используемых данных и удаляет свои объекты. BUSY и PINNED можно повторить после ожидания, но число попыток или общее время должны быть ограничены. NO_VOICE означает, что для нового звука нет свободных голосов: можно пропустить эффект или запланировать его позднее.

Обнуление всего хранилища не является обычной очисткой. Не используйте RESET_STORE или STOP_FX(0) только ради освобождения своих ресурсов: они затрагивают и чужие звуки. Смена session при сбросе разобрана в 6.2.

6.4. Музыка, эффекты и FM вместе

GS и расширенный API используют одну память и один основной источник звука: MOD, PCM или поток. Native PLAY_FX добавляет независимый эффект к основному звуку. Пауза или остановка музыки не останавливает такие эффекты. Исторические direct FX GS могут временно заменять звук первых каналов MOD; это другое поведение, чем независимые native FX.

GS37 удаляет все объекты, созданные через GS. F3/F4 и RESET_STORE затрагивают всё хранилище. OPL2 имеет отдельные регистры: остановка MOD или PCM не снимает ноты FM. Для FM нужен собственный key-off. Уровни микшера GS/native также не регулируют FM; общий выходной регулятор карты действует на итоговый звук.

7. AdLib OPL2

OPL2 постоянно доступен в режиме UltraGS на ISA 0388h (address/status) и 0389h (data); соответствующие ячейки окна - C388h/C389h. CAPS.features bit13 подтверждает поддержку. Запись: выбрать регистр через 0388h, выдержать не менее 3.3 мкс, записать данные в 0389h, выдержать не менее 23 мкс. В OPL_WRITE используются 6 чтений статуса после адреса и 36 после данных, с ограниченными циклами. Поэтому ограничение R3 на чтение статуса касается и этой процедуры. Чтение 0389h возвращает FFh. Нельзя заменять задержку чтением случайного порта или бесконечным poll.

Статус: младшие биты дают 06h; bit6 timer1 overflow, bit5 timer2 overflow, bit7 IRQ summary. Регистры 02/03 - preload таймеров, 04 - управление: bit0/1 запуск timer1/2, bit6/5 маска, bit7 сброс IRQ-флагов. Периоды timer1=(256-preload)*80 мкс, timer2=(256-preload)*320 мкс. Флаг переполнения липкий до reset; masked overflow не поднимает IRQ summary. Для проверки сбросьте 04=80h, задайте 02=FFh, 04=21h, подождите более 80 мкс и проверьте верхние биты C0h; затем остановите и сбросьте таймеры.

Ограничение R3: при включённом BULK alias чтение статуса 0388h надёжно только с Turbo OFF. При выключенном alias его блок обслуживается ранним PIO WAIT. Записи address/data от alias не зависят. Для первого запуска UGOPL используйте `/TURBO:OFF`; подтверждение на реальном стенде R3 остаётся отдельной проверкой. Предыдущий звук стороннего AdLib-теста не доказывает работу таймеров.

Для мелодического инструмента задают пары операторов: 20h multiplier, 40h total level, 60h attack/decay, 80h sustain/release, E0h waveform, C0h feedback/connection. A0h и B0h задают f-number/block; B0 bit5 - key-on. У канала 0 операторные смещения 0 и 3. Key-off снимает bit5 B0, оставляя высоту ноты. Для очистки дождитесь release, а не обрывайте чужие каналы. FM не имеет серверного handle/ownership; приложение должно заранее владеть используемыми каналами. UGOPL TEST/MIX - эксклюзивные диагностические сценарии.

Percussion включается bit5 регистра BDh, биты 0..4 включают hi-hat, cymbal, tom, snare и bass drum, используя каналы 6..8. Это уменьшает число доступных мелодических каналов OPL2, но не PCM-пул. В примере TEST есть проверка percussion.

TEST задаёт инструмент, включает/выключает ноты и таймеры. MIX добавляет собственный MOD8 и четыре mono либо два stereo FX, следит за счётчиками. Ожидаются RESULT PASS, отсутствие новых underruns/PSRAM/protocol errors и возврат собственных объектов. Для фонового FM поверх своего MOD используйте тот же порядок регистровых записей и отдельный key-off, не останавливая MOD.

8. MOD, эффекты, FM и поток: примеры кода

Далее приведены фрагменты работающих DSS-приложений. Они собраны и проверяются вместе с полными программами, на которые даны ссылки. Фрагмент с общими буферами и обработчиками ошибок не является самостоятельным EXE. Команды сборки и запуска сопровождающих примеров находятся в README.

В этих исходниках `RPC` - обёртка над UG_RPC: A содержит opcode, BC - длину запроса в REQ, E - точную ожидаемую длину ответа в RESP. Она проверяет длину и возвращает CF=1 при ошибке. Это другая обёртка, чем строчная `rpc` в главе 2; сам ABI библиотеки остаётся тем же. `RPC_ERROR` и `GS_RPC_ERROR` ведут к общей очистке приложения. `HANDLE` хранит объект, отдельные переменные - номера его запусков. Имена `UG_OP_…` обозначают команды расширенного API, `UG_GS_GS_…` - команды совместимости GS. Их числовые коды приведены в справочнике; в коде удобнее видеть название действия.

8.1. Загрузить MOD через команды GS

Задача: передать готовый музыкальный модуль из памяти Z80 и включить музыку. Ноты затем разбирает карта; программе не нужно подавать их по одной. Если вы пишете новую программу без требований совместимости с GS, можно сразу перейти к варианту расширенного API в разделе 8.3.

Для GS удобна короткая обёртка GS_CALL. Она строит девятибайтовый дескриптор UG_GS_COMMAND: A - команда, B - наличие предварительного байта, C - его значение, E - число читаемых байтов ответа. Post-аргументов у этой обёртки нет; для них заполняют дескриптор непосредственно либо посылают байты через GT_SD. Ответ попадает в GSREPLY. «Предварительный» означает, что этот байт отправляется карте перед кодом команды; это порядок обмена GS, разобранный в главе 4. Внутреннее устройство обёртки приведено ниже. Пользоваться ею можно по четырём параметрам A, B, C, E, не запоминая смещения в GSDESC.

```
GS_CALL:
    ld (GSDESC),a
    ld a,b
    ld (GSDESC+1),a
    ld a,c
    ld (GSDESC+2),a
    xor a
    ld (GSDESC+3),a
    ld a,e
    ld (GSDESC+6),a
    ld hl,GSREPLY
    ld (GSDESC+7),hl
    ld hl,GSDESC
    jp UG_GS_COMMAND
```

В следующем фрагменте MOD уже лежит в памяти Z80 по S8MOD_DATA. Он содержит 2140 байт: заголовок, один четырёхканальный паттерн и собственный короткий образец. Команда 30h начинает загрузку; GT_PENDING получает native handle через LEGACY_INFO и запоминает его для очистки. D1h здесь оставлен как совместимый маркер. Цикл передаёт файл, GT_CLOSE выполняет D2h и проверяет результат, 31h запускает текущий MOD: предварительный байт 0 выбирает именно его. Значение B=1 в вызове означает наличие этого байта, а не номер модуля.

```
    ld a,UG_GS_GS_LOAD_MODULE ; начать загрузку MOD
    call GS_QUERY
    jp c,GS_RPC_ERROR
    call GT_PENDING
    ld a,UG_GS_GS_EXT_D1      ; исторический маркер начала данных
    call GS_SIMPLE
    jp c,GS_RPC_ERROR
    ld hl,S8MOD_DATA
    ld bc,2140              ; размер собственного MOD4 в байтах
.upload:
    push hl
    push bc
    ld a,(hl)
    call GT_SD
```

```
pop bc
pop hl
inc hl
dec bc
ld a,b
or c
jr nz,.upload
call GT_CLOSE
IFDEF UG_GS_TURBO_SLOT
ld hl,S8MOD_DATA
ld bc,2140              ; размер собственного MOD4 в байтах
call GT_READBACK
ENDIF
ld a,UG_GS_GS_PLAY_MODULE ; включить музыку
ld b,1                  ; передать предварительный байт
ld c,0                  ; 0 = текущий MOD
ld e,1                  ; прочитать один байт ответа
call GS_CALL
jp c,GS_RPC_ERROR
```

GT_SD вызывает UG_GS_SD, затем UG_GS_WD; тайм-аут прерывает цикл и ведёт к очистке. Чтение файла с диска вместо массива допускается между такими операциями при закрытом окне. Паузы на чтение не требуют повторного 30h.

Полный сценарий GT_MOD4 также проверяет ноты и громкости каналов. Его исходные данные строит S8TEST_BUILD из ugtestmod8.asm. Это расширенная проверка GS, а последовательность загрузки выше подходит и для обычного проигрывателя.

8.2. Загрузить и запустить GS FX

Задача: воспроизвести короткий звуковой эффект через GS. Сначала разберём загрузку трёх байтов: это маленькая проверка знаковости, не законченная звуковая заставка. Затем посмотрим, как включить повтор загруженного FX.

Загрузка S8 отличается от MOD начальной командой: 3Eh с pre=0. Для U8 можно использовать 38h. В примере переданы три значения S8: -128, 0, 127. У RAW FX нет WAV-заголовка: каждый отправленный байт является образцом.

```
ld a,UG_GS_GS_LOAD_FX_SIGNED ; загрузить FX с выбором формата
ld b,1                  ; есть предварительный байт
ld c,0                  ; формат S8: от -128 до 127
ld e,1                  ; один байт ответа: номер GS-эффекта
call GS_CALL
jp c,GS_RPC_ERROR
call GT_PENDING
ld a,128                ; байт 80h означает -128 в формате S8
call GT_SD
xor a
call GT_SD
ld a,127
call GT_SD
call GT_CLOSE
```

Такой трёхбайтовый образец годится для проверки формата. Для слышимого эффекта нужен более длинный звук или петля. Ниже из того же исходника показан запуск двухбайтового U8 FX с петлёй [0,2): повторяются байты с номерами 0 и 1, байт 2 в петлю не входит. GS_CALL передаёт младший байт границы как pre, два GT_SD - средний и старший. Для границы 2 это байты 02h, 00h, 00h. Команда 80h выбирает выход GS0; pre=0 означает текущий FX. Этот выход соответствует MOD-каналу 0 и временно замещает его звук. Для эффекта поверх MOD без замещения применяйте PLAY_FX из следующего раздела.

```
ld a,UG_GS_GS_LOOP_START ; начало петли
ld b,1                  ; есть предварительный байт
ld c,0                  ; значение предварительного байта
ld e,0                  ; ответных байтов нет
call GS_CALL
jp c,GS_RPC_ERROR
xor a
call GT_SD
```

```
xor a
call GT_SD
ld a,UG_GS_GS_LOOP_END   ; конец петли (не включается)
ld b,1
ld c,2                   ; младший байт: конец петли = 2
ld e,0                   ; ответных байтов нет
call GS_CALL
jp c,GS_RPC_ERROR
xor a
call GT_SD
xor a
call GT_SD
ld a,UG_GS_GS_FX_CH0     ; включить текущий FX на выходе GS0
ld b,1                   ; есть предварительный байт
ld c,0                   ; значение предварительного байта
ld e,0                   ; ответных байтов нет
call GS_CALL
jp c,GS_RPC_ERROR
```

Параметры ноты, громкости, приоритета и поиска свободного канала задаются командами 40h..49h. Команда 39h выбирает канал по маскам SeekFirst/SeekLast; 80h..83h задают его прямо. После команды проверяйте GS_ERROR: квитирование само по себе не гарантирует, что эффект запущен.

Эти GS-примеры работают с собственным набором объектов в изначально пустом хранилище. На выходе GT_CLEANUP заканчивает свой LOAD, проверяет session и удаляет свой набор. GS37 удаляет весь legacy-набор, поэтому включать такую очистку в программу, которая использует чужие GS-объекты, нельзя.

8.3. Запустить MOD8 и добавить mono/stereo FX

Задача: оставить фоновую восьмиканальную музыку и поверх неё включить отдельный эффект. В отличие от direct FX GS, этот эффект не заменяет инструмент в музыкальном канале. В конце прочитаем, какие голоса заняты.

Native-загрузка MOD повторяет главу 2 с двумя изменениями: ALLOC kind=3, а PCM_CONFIG не нужен. Передайте весь MOD через UG_WRITE, увеличивая UG_OFFSET после каждого успешно завершённого блока. После SEAL можно вызвать PLAY_MOD. Именованная константа UG_MOD_LOOP включает повтор песни; поведение команды окончания F00 разобрано в главе 4.

```
call SEAL_BLOB
ld hl,(HANDLE)
ld (REQ),hl
ld a,UG_MOD_LOOP          ; повторять песню
ld (REQ+2),a
ld a,UG_OP_PLAY_MOD
ld bc,UG_OP_PLAY_MOD_REQ_LEN
ld e,UG_OP_PLAY_MOD_RESP_LEN
call RPC
jp c,RPC_ERROR
ld hl,(RESP)
ld (S8PLAY),hl
```

S8PLAY - номер запуска музыки. Для FX создаётся другой RAW-объект; его S16LE-образцы сгенерированы на хосте. S8CH=1 задаёт mono, S8CH=2 - stereo. В обоих случаях ниже загружено 1024 байта: это 512 mono либо 256 stereo кадров. У stereo значения идут попарно L,R. Меняется число каналов, а не количество объектов. **Кадр PCM** - один отсчёт каждого канала: 2 байта для mono S16LE или 4 байта для stereo S16LE. Поэтому 1024 / 4 = 256 stereo-кадров.

PCM_DESC - 24-байтовое описание звука без handle. PCM_CONFIGURE добавляет handle перед ним и посылает PCM_CONFIG. Смещения в листинге относятся к этому описанию: +0 частота, +4 каналы, +5 число бит, +6 формат, +12 длина. Обнуление остальных полей задаёт начало данных 0 и отключает внутреннюю петлю.

```
    ld hl,PCM_DESC
    ld de,PCM_DESC+1
    ld bc,23                ; после первого нуля обнулить ещё 23 байта
    ld (hl),0
    ldir
    ld hl,48000             ; частота в Гц
    ld (PCM_DESC),hl
    ld a,(S8CH)             ; 1 = mono, 2 = stereo
    ld (PCM_DESC+4),a
    ld a,16                 ; 16 бит на отсчёт
    ld (PCM_DESC+5),a
    ld a,2                  ; формат S16LE
    ld (PCM_DESC+6),a
    ld hl,1024              ; длина уже загруженных данных в байтах
    ld (PCM_DESC+12),hl
    call PCM_CONFIGURE
    call SEAL_BLOB
    call S6TEST_PLAY_FX
    jp c,RPC_ERROR
```

S6TEST_PLAY_FX формирует запрос PLAY_FX. UG_VOICE_AUTO (FFFFh) разрешает карте выбрать физические голоса. Priority=128 задаёт средний приоритет эффекта, pan=128 ставит его по центру, flags=2 включает повтор всего объекта. В REQ поля располагаются так: +0 handle, +2 выбор голоса, +4 приоритет, +5 громкость, +6 панорама, +7 флаги. Первые два поля двухбайтовые, остальные занимают по байту: всего восемь байт. В ветке MOD8 используются тихие исходные образцы и gain=64. В проверке заполнения всех 32 голосов gain намеренно снижен до 1. Для приложения обычно достаточно одного выбранного gain в диапазоне 0..64.

```
S6TEST_PLAY_FX:
    call SESSION_CHECK
    ret c
    ld hl,(HANDLE)
    ld (REQ),hl
    ld hl,UG_VOICE_AUTO       ; карта сама выбирает свободные голоса
    ld (REQ+2),hl
    ld a,128                ; приоритет средний, панорама по центру
    ld (REQ+4),a
    ld (REQ+6),a
    ld a,(S8MODE)
    or a
    ld a,1                  ; тихо при проверке сразу 32 голосов
    jr z,.gain
    ld a,64                 ; громче для четырёх тихих образцов MOD8
.gain:
    ld (REQ+5),a
    ld a,2                  ; bit1: повторять весь объект
    ld (REQ+7),a
    ld a,UG_OP_PLAY_FX
    ld bc,UG_OP_PLAY_FX_REQ_LEN
    ld e,UG_OP_PLAY_FX_RESP_LEN
    jp RPC
```

Ответ PLAY_FX содержит instance_id и число занятых голосов. Сохраните ID каждого запуска: STOP_FX принимает его, а FREE - handle исходного объекта. При NO_VOICE эффект не стартовал; музыка продолжает играть. Не останавливайте её ради повторного запуска эффекта. При BUSY дождитесь завершения своей операции с ограничением числа повторов.

Чтобы прочитать состояние, сначала получите снимок. В тестовом приложении S6TEST_SNAPSHOT копирует все 64 байта в S6SNAP: последующие запросы используют RESP и не должны затереть ключ снимка.

```
S6TEST_SNAPSHOT:
    ld a,UG_OP_SNAPSHOT
    ld bc,UG_OP_SNAPSHOT_REQ_LEN
    ld e,UG_OP_SNAPSHOT_RESP_LEN
    call RPC
    ret c
    ld hl,RESP
    ld de,S6SNAP
    ld bc,UG_OP_SNAPSHOT_RESP_LEN
    ldir
    ret
```

Следующий запрос читает одну запись физического голоса: 20 байт заголовка плюс 20 байт VOICEREC. Первые восемь байт запроса - session, следующие четыре - номер снимка. Эти два значения защищают от смешения данных, полученных в разные моменты. S8INDEX выбирает запись голоса, начиная с 0; поле +12 передаёт её номер, +14 - сколько записей прочитать. Для следующей страницы увеличивайте first на возвращённый count, сохраняя тот же ключ. При STATE получите новый SNAPSHOT и начните обход заново.

```
    ld hl,S6SNAP+4            ; session из снимка
    ld de,REQ
    ld bc,8                   ; размер session
    ldir
    ld hl,S6SNAP
    ld bc,4                   ; номер снимка: 4 байта
    ldir
    ld a,(S8INDEX)
    ld l,a
    ld h,0
    ld (REQ+12),hl
    ld hl,1                   ; запросить одну запись голоса
    ld (REQ+14),hl
    ld a,UG_OP_VOICE_STATUS
    ld bc,UG_OP_VOICE_STATUS_REQ_LEN
    ld e,40                   ; заголовок 20 + одна запись 20 байт
    call RPC
    jp c,RPC_ERROR
```

Полный S8TEST_MOD / S8TEST_PHASE проверяет MOD8, четыре mono либо два stereo FX, паузу и чтение данных во время звука. Его очистка останавливает свои instances и освобождает записанные handles; она не использует STOP_FX(0).

8.4. Добавить FM поверх MOD

FM не требует загрузки образцов. Достаточно задать инструмент через OPL2, включить key-on и позднее снять его. MOD при этом продолжает играть: его команды и ресурсы независимы от FM.

Задача: добавить синтезированную ноту, пока MOD играет своим ходом. В OPL2 вместо массива звуковых отсчётов программа задаёт инструмент и высоту ноты. Key-on означает «нажать клавишу», key-off - «отпустить».

Процедура ниже принимает A=номер регистра, E=значение и сохраняет регистры. OPL_OPEN/OPL_CLOSE управляют ISA-окном. В вызывающем приложении предварительно заданы SLOT и OPLTURBO=0: из-за ограничения R3 чтения статуса должны идти без Turbo при включённом BULK alias. C388h и C389h получаются из начала ISA-окна C000h и портов OPL2 0388h/0389h. Числа 6 и 36 в циклах чтения задают задержки после выбора регистра и записи данных, указанные в главе 7.

```
OPL_WRITE:
    push af
    push bc
    push de
    push hl
    push af
    call OPL_OPEN
    pop af
    ld (0xC388),a
    ld b,6
.address_delay:
```

```
    ld a,(0xC388)
    djnz .address_delay
    ld a,e
    ld (0xC389),a
    ld b,36
.data_delay:
    ld a,(0xC388)
    djnz .data_delay
    call OPL_CLOSE
    pop hl
    pop de
    pop bc
    pop af
    ret
```

Далее - реальная настройка инструмента и включение нот в тесте OPL_RUN. Массив opl_operators содержит смещения операторов всех девяти каналов. Для одного канала 0 нужны только операторы 0 и 3; значения регистров остаются теми же. Предварительно OPL_RUN включает waveform select (01h=20h), выбирает мелодический режим (BDh=0), снимает старые key-on и задаёт C0h..C8h=0. Оператор - один генератор синтезатора; инструмент использует два оператора. Отсюда 18 операторов для 9 каналов в диагностическом примере.

У инструмента нет единственного «правильного» набора чисел: они задают тембр. В этом примере использованы следующие настройки (в таблице числа с h шестнадцатеричные):

Группа регистров	Значение	Что задаёт
20h + смещение оператора	21h	Множитель частоты 1 и удержание уровня
40h + смещение	18h	Ослабление уровня оператора на 18 дБ
60h + смещение	F3h	Быстрая атака F и спад 3
80h + смещение	35h	Уровень удержания 3 и затухание 5
E0h + смещение	00h	Синусоидальная форма волны
C0h + номер канала	00h	FM-связь операторов, без обратной связи
A0h/B0h + номер канала	Из массива частот / 31h	Высота ноты и включение звука

```
    ld hl,opl_operators
    ld b,18
.operators:
    ld a,(hl)
    add a,0x20
    ld e,0x21
    call OPL_WRITE
    ld a,(hl)
    add a,0x40
    ld e,24
    call OPL_WRITE
    ld a,(hl)
    add a,0x60
    ld e,0xF3
    call OPL_WRITE
    ld a,(hl)
    add a,0x80
    ld e,0x35
    call OPL_WRITE
    ld a,(hl)
    add a,0xE0
    ld e,0
    call OPL_WRITE
    inc hl
    djnz .operators
    ld b,9
    ld a,0xA0
```

```
    ld hl,opl_frequencies
.notes:
    ld e,(hl)
    inc hl
    call OPL_WRITE
    add a,0x10
    ld e,0x31
    call OPL_WRITE
    sub 0x0F
    djnz .notes
```

40h получает десятичное 24, то есть 18h, а не 24h. A0h задаёт младшую часть частоты, B0h=31h - её старшие биты, октаву и key-on. Для канала 0 в этом инструменте key-off соответствует записи B0h=11h: очищен только бит 5. Снятие ключа запускает release огибающей; оно не останавливает MOD.

В OPL_RUN / OPL_CLEANUP диагностическое приложение владеет всеми девятью каналами. В своём движке назначайте FM-каналы явно и снимайте ключи только у своих нот. Протокол OPL2 не возвращает handle и не хранит сведения о владельце. Процедура OPL_CLEANUP для эксклюзивной диагностики не подходит для завершения одного инструмента среди чужих.

8.5. Подавать WAV с диска небольшими блоками

Задача: проиграть файл, который не хочется целиком хранить на карте. Программа читает его частями и пополняет запас звука быстрее, чем карта успевает этот запас израсходовать. Этот вариант сложнее главы 2; для коротких эффектов проще загрузить объект целиком.

При streaming в PSRAM хранится кольцевой буфер, а не весь файл. Сначала хост разбирает RIFF: находит fmt и data, проверяет PCM-формат и размер кадра. В кольцо передаётся только содержимое data, без заголовков WAV. STREAM_WAV учитывает неизвестные chunks и их выравнивание.

STREAM_CREATE получает первые восемь байт PCM_DESC и три u32 размера: BUFFER, PREFILL, LOW. Все размеры в байтах. Заполненное кольцо не должно пересекать границы PCM-кадров. Handle сохраняется для STATE, FEED, EOF и CLOSE.

Например, для U8 mono 8000 Гц можно выделить BUFFER=16384 байта, начинать звук после PREFILL=8192 байт и запрашивать пополнение при LOW=4096 байт. Это примерно две секунды запаса, одна секунда перед стартом и полсекунды до опустошения при достижении LOW. В исходнике SBUFFER хранит эти три четырёхбайтовых размера подряд, поэтому вторая LDIR копирует 12 байт.

```
STREAM_CREATE:
    call SESSION_CHECK
    jp c,RPC_ERROR
    ld hl,PCM_DESC
    ld de,REQ
    ld bc,8                 ; частота и формат PCM
    ldir
    ld hl,SBUFFER
    ld bc,12                ; три размера по 4 байта
    ldir
    ld a,UG_OP_STREAM_CREATE
    ld bc,UG_OP_STREAM_CREATE_REQ_LEN
    ld e,UG_OP_STREAM_CREATE_RESP_LEN
    call RPC
    jp c,RPC_ERROR
    ld hl,(RESP)
    ld (HANDLE),hl
    ld a,1
    ld (SOWN),a
    ret
```

После STREAM_START карта ждёт PREFILL. Хост читает порцию файла при закрытом ISA-окне, выбирает N не больше 1024 и доступного free, затем передаёт её. READ файла и FEED - две отдельные операции. Нельзя читать диск, оставив окно открытым или рассчитывая, что callback DSS сохранит его отображение.

В STREAM_FEED переменные HANDLE, SSEQ, BLOCK и SREADPTR содержат handle, номер пакета, число байтов и адрес уже прочитанного блока. SSEQ начинается с 1. VERIFIED добавляет CRC данных. Это

фрагмент приложения UGPLAY: его UG_RPC собран с APP_PLAY и на WRITE_GRANT вызывает STREAM_BULK **до** чтения окончательного ответа. У обычной сборки библиотеки такого вызова нет.

Ниже показано формирование одного пакета и его передача. Смещения +0, +2, +6, +8 означают handle (2 байта), номер пакета (4), длину данных (2) и флаги (1). Получается 9 байт аргументов. При VERIFY добавляются четыре байта контрольной суммы, итого 13; значение 128 включает bit7 флагов VERIFIED. SSENT отмечает, переданы ли данные, а UGTM.NOESC откладывает опрос клавиши до окончания пакета, чтобы не оборвать обмен посередине.

```
 STREAM_FEED:
     ld hl,(HANDLE)
     ld (REQ),hl
     ld hl,SSEQ
     ld de,REQ+2
     ld bc,4
     ldir
     ld hl,(BLOCK)
     ld (REQ+6),hl
     xor a
     ld (REQ+8),a
     ld (SSENT),a
     ld bc,9
     ld a,(VERIFY)
     or a
     jr z,.send
     ld hl,(SREADPTR)
     ld bc,(BLOCK)
     call UG_CRC32
     ld hl,UGCRC.OUT
     ld de,REQ+9
     ld bc,4
     ldir
     ld a,128
     ld (REQ+8),a
     ld bc,13
 .send:
     ld a,1
     ld (UGTM.NOESC),a ; keyboard only after the complete packet
     ld a,UG_OP_STREAM_FEED
     ld e,12
     call RPC
     push af
     xor a
     ld (UGTM.NOESC),a
     pop af
```

После последнего блока отправьте STREAM_EOF и опрашивайте STREAM_STATE до state=3. Затем STREAM_CLOSE(handle) освобождает своё кольцо; при ошибке или ESC закрываются и поток, и файловый дескриптор DSS. Ошибка чтения файла не является EOF: её нельзя молча превращать в успешное короткое воспроизведение.

В полном STREAM_FEED после показанного фрагмента проверяется результат вызова: подтверждённый блок увеличивает SSEQ, ошибка ведёт к восстановлению. Потерянный ответ FEED не означает, что данные пропали. Код сверяет session и next_sequence через STREAM_STATE, не дублируя пакет. Если подтверждения нет, он завершает сценарий с ошибкой. Учебный WAV - собственный U8 mono 8000 Гц; ограничения непрерывной подачи больших файлов описаны в 5.4.

9. Приложения

9.1. Карта портов и биты возможностей

Адреса ячеек ниже рассчитаны для BASE=02A0h и открытого ISA-окна. SNAP_INDEX допускает 0..63; чтение за концом - UNDERFLOW, без wrap.

Смещ.	ISA / Z80	Регистр	Чтение	Запись
00	02A0 / C2A0	BULK_DATA	bulk_byte	bulk_byte
01	02A1 / C2A1	X_STATUS	status	bad_register
02	02A2 / C2A2	X_CONTROL	zero	control
03	02A3 / C2A3	X_REQUEST	response_byte	request_byte
04	02A4 / C2A4	SNAP_INDEX	index	index
05	02A5 / C2A5	SNAP_DATA	snap_byte	bad_register
06	02A6 / C2A6	X_ERROR	xerror	bad_register
07	02A7 / C2A7	GS_ERROR	gserror	bad_register
08	02A8 / C2A8	API_MAJOR	const	bad_register
09	02A9 / C2A9	API_MINOR	const	bad_register
0A..12	02AA / C2AA	RSVD_0A	zero	bad_register
13	02B3 / C2B3	GS_DATA	gs_data_out	gs_data_in
14..1A	02B4 / C2B4	RSVD_14	zero	bad_register
1B	02BB / C2BB	GS_CMDSTAT	gs_status	gs_command
1C..1F	02BC / C2BC	RSVD_1C	zero	bad_register
-	01D0..01D2 / C1D0..C1D2	CONFIG	selector/data	unlock/select/data
-	00B3 / C0B3	GSALIAS DATA	GS out	GS in
-	00BB / C0BB	GSALIAS CMD/STATUS	status	command
-	0388 / C388	OPL2	status	address
-	0389 / C389	OPL2 data	FFh	value

X_STATUS бит	Название
0	READY
1	BUSY
2	REPLY
3	WRITE_GRANT
4	READ_GRANT
5	XERR
6	GSERR
7	AUDIO

CAPS.features бит	Значение
0	GS musical
1	Slow read/write через GS DATA
2	BULK read/write
3	BULK alias включён
4	Playlist: сейчас 0, QUEUE не реализованы
5	Согласованные snapshots
6	CRC32 verification
7	MEM_TEST
8	Независимые FX
9	GSALIAS включён
10	Stereo FX
11	PCM streaming
12	Явная политика MOD F00
13	Независимый OPL2

CAPS.formats: bit0 RAW8, bit1 RAW16, bit2 WAV_PCM, bit3 MOD4/FLT4, bit4 MOD8_CHN, bit5 FLT8 PCM, bit6 MOD1..32. Неизвестные биты не используйте.

9.2. Ошибки и действия программиста

Код hex	Ошибка	Действие
00	OK	Операция завершена; проверить длину ответа.
01	BAD_OPCODE	Проверить API/CAPS; не использовать резерв QUEUE.
02	BAD_ARGUMENT	Проверить флаги, reserved, gain и параметры.
03	LENGTH	Исправить длину payload/число данных, сбросить FAULT при необходимости.
04	CRC	Не использовать повреждённый ответ; сверить результат изменяющей команды.
05	BUSY	Дождаться конца своей операции с deadline; не сбрасывать чужие ресурсы.
06	STATE	Проверить состояние/ключ snapshot/READY, начать новый snapshot при stale key.
07	HANDLE	Проверить session и принадлежность handle/ID.
08	RANGE	Проверить offset, N, номер страницы, order/row.
09	NO_MEMORY	Проверить free и largest; освободить свои объекты либо уменьшить запрос.
0A	NO_HANDLES	Лимит объектов или выдач handles; освободить свои объекты, при исчерпании session требуется согласованный reset.
0B	FORMAT	Проверить поддержанный формат и целостность файла.
0C	NO_VOICE	Отложить звук; не останавливать чужую музыку.
0D	TIMEOUT	Закрыть окно, выполнить ограниченное восстановление и сверку состояния.
0E	OVERFLOW	Не передавать лишние байты; проверить исчерпание token/sequence.
0F	UNDERFLOW	Не читать FIFO без REPLY/READ_GRANT; восстановить framing.
10	SHORT_TRANSFER	Додать нельзя после отмены: начать корректный блок заново.
11	PSRAM	Остановить сценарий; сохранить диагностику памяти.
12	PINNED	Остановить свои instances и дождаться снятия pins.
13	UNSUPPORTED_MEMORY	Память платы не поддержана; проверить аппаратную конфигурацию.
14	BAD_REGISTER	Исправить адрес/направление доступа.
15	CANCELLED	Операция отменена; завершить очистку собственных ресурсов.
16	INTERNAL	Сохранить диагностику; не продолжать изменение store.

GS_ERROR: 10h нехватка памяти, 11h нехватка legacy/общих handles; PSRAM отображается как E1h, SHORT_TRANSFER как E0h, остальные коды совпадают с native, если нет конфликта. F0 не очищает ошибку. 00 вне LOAD, F3/F4 и CLEAR_ERRORS очищают. Не путайте эти значения с кодами выхода DSS-утилит: например host TIMEOUT печатается как code8, ESC как code10.

9.3. Утилиты

Утилита	Назначение и типичная команда
UGCFG	INFO /DIAG, ULTRAGS, PORTS, GSALIAS, SAVE, VERIFY
UGMEM	INFO, TEST QUICK/FULL, ROUNDTRIP N /VERIFY, FREE handle
UGLOAD	файл /TYPE:RAW или WAV или MOD; возвращает handle
UGDUMP	handle файл /VERIFY; byte-exact выгрузка
UGPLAY	файл /WAIT, HANDLE h, FXHANDLE h, PAUSE/RESUME/STOP, STREAM файл
UGTEST	PCM, FX, MIX, GS, MOD4, MOD8, FLT8; собственные fixtures
UGMON	/COUNT:N; summary и голоса
UGOPL	TEST, MIX /FX:MONO или STEREO; FM диагностика
UGPF	Ускоренная BULK-подача UGPLAY; ограничения streaming сохраняются
UGBENCH / UGDIAG	Измерения обмена и диагностические сценарии

Синтаксис и доступные опции смотрите через `/?`. Числа handle/instance из вывода - значения конкретной session, не постоянные номера файлов. UGLOAD оставляет объект для следующих программ; не ожидайте автоматического FREE при выходе. UGPLAY без WAIT может оставить звук после возврата в DSS.

9.4. Совместимость и расхождения источников

Поддержаны RAW U8/S8/S16LE, обычный PCM WAV и MOD с 1-32 каналами: M.K., M!K!, FLT4, 8CHN, FLT8, 1CHN..9CHN и 10CH..32CH. FLT8 использует парные паттерны; это не переименованный 8CHN. S3M, XM, IT, MP3, сжатый WAV, float и StarTrekker AM/FM companions не поддерживаются. Выход карты: 48000 Гц, signed 16-bit stereo, линейная интерполяция.

Название версии	Что оно обозначает
GS 1.04 / дополнения 1.05	Исторические музыкальные команды; CAPS.gs_profile=1
UltraGS API 1.0 / VLS1	Текущий расширенный интерфейс; INFO длиной 40 байт, UGS1 и VLS1
PicoGUS protocol 4	Конфигурационные порты: режим, адреса, SAVE; magic=DDh
Mode 8	Режим UltraGS в комплекте прошивок; не номер слота Sprinter

Вопрос	Действующий контракт
Внутренний Z80 GS/NeoGS, RAM, DAC, MP3/SD	Не эмулируются; только музыкальный профиль
GS 08, 21, 3E	No-op; free RAM; 0=S8/1=U8 соответственно
GS32 и F00	Пауза с сохранением ресурсов; FX продолжаются
Старые §6-7 T3: INFO36/CAPS32/SNAPSHOT128	Заменены решением VLS1: 40/44/64
Старые voice_mask и фиксированные 8+4 lanes	Заменены instance IDs и shared pool32
QUEUE в схеме и цели финального MVP	Резерв; BAD_OPCODE, feature4=0
CAPS.playlist_entries const32 в wire-schema	Реальная прошивка возвращает 0; отмеченное исключение схемы

Вопрос	Действующий контракт
GSALIAS только runtime в старых README	SAVE сохраняет ON/OFF в текущей прошивке
MOD8/FLT8/многоканальные MOD	Реализованы; не переносить старый список «будущих» форматов
Непрерывный 44100/S16 stereo с диска	Не подтверждён, есть аппаратный REAL_FAIL
OPL status с BULK alias и Turbo	R3 требует Turbo OFF для status read

9.5. Словарь

Object - хранимые исходные байты и метаданные. Handle - u16 ссылка на object в session. Instance - отдельный запуск. Voice - физический PCM-голос. Foreground - единственный основной MOD/PCM/stream. FX - независимый PCM-запуск. Pin - запрет удаления используемого объекта. Grant - разрешение передать ровно один блок. SEAL - проверка и публикация объекта. Frame - либо PCM-кадр, либо управляющий кадр протокола; контекст всегда указан. Session - эпоха store, меняется при reset. Snapshot - согласованная копия состояния. HOST - модельная проверка; REAL - подтверждённый опыт на плате.

9.6. Источники и авторы

Порядок объяснения «характеристики - устройство - интерфейс - команды - примеры» опирается на исторические руководства. Текст данного руководства написан для UltraGS с учётом отличий от GS и NeoGS.

- General Sound v1.04, редакция 006, с дополнениями ROM 1.05: STINGER, Cyrax, Inc.; перекодировка Evgeny Muchkin, оформление документа CHRV (24.02.2012).
- NeoGS. Руководство программиста, version 0.1.A: NedoPC, 2009; документ помечен fpgaD 03.11.2008. Индивидуальный автор на титульном листе не указан.
- T3 UltraGS: основной источник требований; при чтении учитывать последующие решения VLS1, streaming, F00, GSALIAS и OPL2.
- Wire schema, диспетчер native, диспетчер GS, библиотека DSS: проверка действующего поведения.

Ссылки на локальные исходники относятся к этому репозиторию. Описанное поведение сверено с текущими диспетчерами; расхождения с историческими документами перечислены в 9.4.

9.7. Форматы команд UltraGS API 1.0

В таблицах смещения считаются от начала полезных данных запроса или ответа, без двухбайтового заголовка кадра. Все многобайтовые числа little-endian, без выравнивания. u8/u16/u32/u64 - беззнаковые числа, s8 - знаковое. `byte[N]` - N байт, `FXREC[]` и `VOICEREC[]` - записи переменного количества. Именованные структуры PCM_DESC и PAGE_HEADER описаны в 9.8. Успешный ответ имеет status=0. При ошибке сначала обрабатывайте status, а не интерпретируйте буфер как успешный ответ.

00h INFO

Определяет версию и состояние карты. До ALLOC/PLAY проверьте UGS1, API 1.0, VLS1 и boot_state=0; сохраните session. Лишние байты запроса дают BAD_OPCODE. Команда доступна даже при ошибке памяти.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: INFO (40 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	magic	byte[4]	4	значение UGS1
4	api_major	u8	1	значение 1
5	api_minor	u8	1	значение 0
6	mode	u8	1	значение 8
7	boot_state	u8	1	-
8	fw_version	u32	4	-
12	session	u64	8	-
20	build_id	byte[16]	16	-
36	layout	byte[4]	4	значение VLS1

Разрешена во время передачи блока.

01h CAPS

Возвращает доступные функции, форматы и пределы. Проверяйте нужные биты перед использованием функции. music_limit и fx_limit относятся к общему пулу и не складываются. Во время передачи блока возвращает BUSY.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: CAPS (44 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	features	u32	4	-
4	formats	u32	4	-
8	output_hz	u32	4	-
12	max_block	u16	2	значение 1024
14	max_payload	u16	2	значение 128
16	objects	u16	2	значение 128
18	playlist_entries	u16	2	Фактически 0 (схема const32 устарела; очереди не реализованы)
20	total_voices	u16	2	-
22	music_limit	u16	2	-
24	fx_limit	u16	2	-
26	gs_profile	u8	1	-
27	pool_kind	u8	1	значение 1
28	base	u16	2	-
30	alias	u16	2	-
32	pcm_rate_min	u32	4	значение 8000

Смещ.	Поле	Тип	Байт	Примечание
36	pcm_rate_max	u32	4	значение 48000
40	voice_record_size	u8	1	значение 20
41	fx_record_size	u8	1	значение 16
42	snapshot_size	u16	2	значение 64

02h MEM_INFO

Возвращает размеры памяти в байтах и число объектов. Для одного ALLOC нужен непрерывный участок largest. Не меняет память и не останавливает звук.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: MEM_INFO (20 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	physical	u32	4	-
4	pool	u32	4	-
8	free	u32	4	-
12	largest	u32	4	-
16	objects	u16	2	-
18	limit	u16	2	-

Разрешена во время передачи блока.

03h STATS

Читает 16 счётчиков u32. max_render_us и max_command_us измеряются в микросекундах; rx_high_water и tx_high_water - пиковое заполнение FIFO в байтах. Остальные поля считают операции, байты, блоки или отсчёты по имени. Для наблюдения за своим сценарием сравнивайте значения до и после него.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: STATS (64 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	isa_reads	u32	4	-
4	isa_writes	u32	4	-
8	upload_bytes	u32	4	-
12	download_bytes	u32	4	-
16	write_blocks	u32	4	-
20	read_blocks	u32	4	-
24	crc_errors	u32	4	-
28	protocol_errors	u32	4	-

Смещ.	Поле	Тип	Байт	Примечание
32	transfer_timeouts	u32	4	-
36	audio_underruns	u32	4	-
40	psram_errors	u32	4	-
44	max_render_us	u32	4	-
48	max_command_us	u32	4	-
52	rx_high_water	u32	4	-
56	tx_high_water	u32	4	-
60	clip_samples	u32	4	-

Разрешена во время передачи блока.

04h CLEAR_STATS

Обнуляет общую статистику. Звук и объекты сохраняются. Команда влияет и на измерения других наблюдателей; для обычного мониторинга предпочтительнее разность двух STATS.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK.

Нет полезных данных (0 байт).

05h MEM_TEST

mode=0: короткая проверка, mode=1: полная. Требует пустого хранилища; при занятых ресурсах BUSY. При ошибке PSRAM результат теста всё равно содержит tested, first_bad и expected/actual. На успехе first_bad=FFFFFFFFh.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	mode	u8	1	-

Ответ при OK: MEM_TEST_RESP (14 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	tested	u32	4	-
4	first_bad	u32	4	-
8	expected	u8	1	-
9	actual	u8	1	-
10	elapsed_ms	u32	4	-

06h RESET_STORE

Глобальное обслуживание с ключом 55475253h: level=0 очищает объекты и воспроизведение, level=1 дополнительно проверяет память. Возвращает новую session; все старые handles и IDs недействительны. Это не очистка отдельной программы. При незавершённой передаче или GS LOAD возможен BUSY.

Запрос: RESET_STORE_REQ (5 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	key	u32	4	значение 0x55475253
4	level	u8	1	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	session_id	u64	8	-

10h ALLOC

Создаёт UPLOADING-объект kind=0 BLOB, 1 RAW, 2 WAV или 3 MOD. size задаётся в байтах, положительный. Сохраняйте возвращённый handle; reserve - фактический объём с округлением до 16. NO_MEMORY означает нехватку непрерывной памяти, NO_HANDLES - лимит объектов или номеров.

Запрос: ALLOC_REQ (5 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	kind	u8	1	-
1	size	u32	4	-

Ответ при OK: ALLOC_RESP (6 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	reserved	u32	4	Фактический резерв в байтах, округлён до 16

11h FREE

Удаляет указанный объект. Сначала остановите все собственные запуски, использующие его. PINNED означает, что объект ещё нужен проигрывателю или передаче; повтор допускается после ожидания. Неизвестный handle даёт HANDLE.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK.

Нет полезных данных (0 байт).

12h OBJECT_INFO

Читает метаданные объекта: state=0 загрузка, 1 готов, 2 испорчен; pins запрещает удаление. size и uploaded измеряются в байтах. Параметры звука имеют смысл после PCM_CONFIG или успешного разбора SEAL. Неизвестный объект даёт HANDLE.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK: OBJECT_INFO (38 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	kind	u8	1	-
3	state	u8	1	-
4	pins	u16	2	-
6	size	u32	4	-
10	uploaded	u32	4	-
14	crc	u32	4	-
18	data_offset	u32	4	-
22	data_length	u32	4	-
26	rate	u32	4	-
30	frames	u32	4	-
34	channels	u8	1	-
35	bits	u8	1	-
36	sample_format	u8	1	-
37	mod_channels	u8	1	-

Разрешена во время передачи блока; только для объекта этой передачи.

13h SEAL

Проверяет полностью загруженный объект и переводит его в READY. Для RAW предварительно нужен PCM_CONFIG. flags=0 требует expected_crc=0; flags=1 проверяет CRC всего объекта. Успех возвращает actual_crc. STATE означает неподходящую фазу; FORMAT/CRC - неверное содержимое. Не повторяйте SEAL объекта READY.

Запрос: SEAL_REQ (7 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	flags	u8	1	Бит 0: проверить CRC всего объекта
3	expected_crc	u32	4	-

Ответ при OK: SEAL_RESP (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	actual_crc	u32	4	-

14h PCM_CONFIG

Задаёт описание только RAW в состоянии UPLOADING, без pins. Вложенная структура PCM_DESC приведена в 9.8. Неправильные частота, формат, длина или петля дают FORMAT; другой kind или состояние - STATE.

Запрос: PCM_CONFIG_REQ (26 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	desc	PCM_DESC	24	-

Ответ при OK.

Нет полезных данных (0 байт).

15h OBJECT_ENUM

Обходит handles по возрастанию: первый запрос after_handle=0, следующий - с ранее возвращённым next_handle. Ноль в ответе завершает обход. Перечисление само по себе не доказывает, что объект принадлежит вызывающей программе.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	after_handle	u16	2	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	next_handle	u16	2	-

16h LEGACY_INFO

Возвращает native handles текущих GS MOD, FX и незавершённой загрузки; 0 означает отсутствие. uploaded - число принятых байтов LOAD. Эти u16 handles отличаются от однобайтовых номеров GS.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: LEGACY_INFO_RESP (10 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	module	u16	2	-
2	fx	u16	2	-
4	pending	u16	2	-
6	uploaded	u32	4	-

Разрешена во время передачи блока.

20h WRITE_BEGIN

Начинает запись N=1..1024 байт. Для последовательной загрузки offset=uploaded; BLOB также допускает перезапись уже загруженного диапазона. transport=00h/01h FAST или 80h/81h VERIFIED. Сначала получите token и WRITE_GRANT, затем отправьте данные. STATE/RANGE означают неподходящую фазу или диапазон.

Запрос: WRITE_BEGIN_REQ (9 байт FAST; 13 байт VERIFIED).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	offset	u32	4	-
6	length	u16	2	-
8	transport	u8	1	-
9	crc	u32	4	VERIFIED: полный размер 13 байт; только VERIFIED

Ответ при OK: WRITE_BEGIN_RESP (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-

21h WRITE_END

Передаёт token после всех N байт. Только успешный END подтверждает запись. SHORT_TRANSFER означает недостачу данных, CRC - несовпадение в VERIFIED. После отказа блок не считается подтверждённым; сверяйте OBJECT_INFO перед продолжением.

Запрос: WRITE_END_REQ (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-

Ответ при OK.

Нет полезных данных (0 байт).

Разрешена во время передачи блока.

22h READ_BEGIN

Начинает чтение N=1..1024 байт в пределах uploaded; объект может быть READY и играть. После token дождитесь READ_GRANT. В VERIFIED ответ дополнительно содержит CRC данных. Запрос CRC не содержит. RANGE означает чтение за доступной границей.

Запрос: READ_BEGIN_REQ (9 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	offset	u32	4	-
6	length	u16	2	-
8	transport	u8	1	-

Ответ при OK: READ_BEGIN_RESP (4 байт FAST; 8 байт VERIFIED).

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-
4	crc	u32	4	VERIFIED: полный размер 8 байт; только VERIFIED

23h READ_END

Завершает чтение по token. В VERIFIED передайте вычисленную host_crc; в FAST поле отсутствует. Неверный token даёт HANDLE, неполное чтение - SHORT_TRANSFER, несовпадение CRC - CRC. Данные объекта не меняются.

Запрос: READ_END_REQ (4 байт FAST; 8 байт VERIFIED).

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-
4	host_crc	u32	4	VERIFIED: полный размер 8 байт; только VERIFIED

Ответ при OK.

Нет полезных данных (0 байт).

Разрешена во время передачи блока.

24h TRANSFER_CANCEL

Отменяет передачу с данным token, освобождая её временный буфер и pin. Ранее завершённые блоки сохраняются. Неизвестный или уже завершённый token даёт HANDLE.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	transfer	u32	4	-

Ответ при OK.

Нет полезных данных (0 байт).

Разрешена во время передачи блока.

30h PLAY_PCM

Запускает READY RAW/WAV как основной звук. repeats=0 повторяет весь объект, 1 играет один раз, 2..65535 задаёт число проходов; внутренняя RAW-петля требует repeats=1. gain=0..64, pan=0..255. Возвращает play_id. При NO_VOICE/BUSY новый запуск не принят; прежний звук сохраняется.

Запрос: PLAY_PCM_REQ (6 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	repeats	u16	2	-
4	gain	u8	1	-
5	pan	u8	1	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-

31h PLAY_MOD

Запускает READY MOD с начала и возвращает play_id. flags: 1 LOOP, 2 IGNORE_F00, 4 PAUSE_F00; последние два несовместимы. Без них действует GS6A. Некорректные flags дают BAD_ARGUMENT, нехватка голосов - NO_VOICE, неподходящий объект - STATE/FORMAT.

Запрос: PLAY_MOD_REQ (3 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	flags	u8	1	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-

32h CONTROL

Управляет основным звуком: action=0 STOP, 1 PAUSE, 2 RESUME. play_id=0 выбирает текущий запуск, поэтому для очистки используйте точный ID. RESUME требует паузы. Нет подходящего активного запуска - STATE. STOP не удаляет обычный объект, но закрывает поток.

Запрос: CONTROL_REQ (3 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-
2	action	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

33h SEEK_MOD

Переводит активный MOD на order и row=0..63. order меньше длины песни. play_id=0 означает текущий MOD; неверный ID или не MOD дают STATE, неверная позиция - RANGE. Эффекты и темп сбрасываются на 6/125, состояние паузы сохраняется.

Запрос: SEEK_MOD_REQ (5 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-
2	order	u16	2	-
4	row	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

34h SET_MIX

Четыре уровня 0..64: music_master, fx_master, music_gain, fx_gain. Изменяет общие уровни основного звука и FX, включая GS-эффекты; OPL2 не затрагивает. Значение выше 64 даёт BAD_ARGUMENT. Это общая настройка, без ID отдельного запуска.

Запрос: SET_MIX_REQ (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	music_master	u8	1	-
1	fx_master	u8	1	-
2	music_gain	u8	1	-
3	fx_gain	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

35h SET_TEMPO

Меняет текущий MOD: speed=1..31 тиков на строку, BPM=32..255. Нет активного MOD - STATE, неверные значения - BAD_ARGUMENT. Последующие команды темпа внутри песни могут снова изменить значения.

Запрос: SET_TEMPO_REQ (2 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	speed	u8	1	-
1	bpm	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

36h PLAY_FX

Создаёт независимый запуск READY RAW/WAV. preferred_voice=FFFFh выбирает голоса автоматически, иначе 0..31. priority=1..254, gain=0..64, pan=0..255. flags: 1 разрешает вытеснять FX меньшего приоритета, 2 повторяет весь объект. Музыка не вытесняется; при NO_VOICE ни один канал эффекта не запускается.

Запрос: PLAY_FX_REQ (8 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	preferred_voice	u16	2	-
4	priority	u8	1	-
5	gain	u8	1	-
6	pan	u8	1	-
7	flags	u8	1	-

Ответ при OK: PLAY_FX_RESP (4 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	instance_id	u16	2	-
2	voice_count	u16	2	-

37h STOP_FX

Останавливает instance_id эффекта, не удаляя его объект. ID=0 останавливает все native FX; для очистки одного приложения он не подходит. Несуществующий ненулевой ID даёт HANDLE. Музыку и GS direct FX команда не останавливает.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	instance_id	u16	2	-

Ответ при OK.

Нет полезных данных (0 байт).

38h SET_PAN

Меняет панораму логического MOD-канала, а не физического голоса. channel должен существовать и быть 0..7 даже у MOD с большим числом каналов; иначе RANGE. pan=0..255. Нет активного MOD - STATE. Настройку могут заменить PLAY, SEEK и команды самого MOD.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	logical_channel	u16	2	-
2	pan	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

39h CONTROL_FX

Управляет одним ненулевым instance_id: action=0 STOP, 1 PAUSE, 2 RESUME. При паузе голоса и позиция сохраняются. Нет такого FX - HANDLE, RESUME без паузы - STATE. Основной звук и другие FX продолжаются.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	fx_id	u16	2	-
2	action	u8	1	-

Ответ при OK.

Нет полезных данных (0 байт).

40h QUEUE_CLEAR (резерв)

Не реализована: BAD_OPCODE, полезных данных успешного ответа нет.

41h QUEUE_ADD (резерв)

Не реализована: BAD_OPCODE, полезных данных успешного ответа нет.

42h QUEUE_PLAY (резерв)

Не реализована: BAD_OPCODE, полезных данных успешного ответа нет.

43h QUEUE_INFO (резерв)

Не реализована: BAD_OPCODE, полезных данных успешного ответа нет.

44h STREAM_CREATE

Создаёт единственное PCM-кольцо. rate=8000..48000, channels=1/2, bits=8/16, формат U8/S8/S16LE; reserved=0. Размеры в байтах, кратные размеру PCM-кадра; buffer>0, 0<prefill<=buffer, 0<=low<prefill. Второе кольцо даёт BUSY; неверные размеры - BAD_ARGUMENT, формат - FORMAT.

Запрос: STREAM_CREATE_REQ (20 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	rate	u32	4	-
4	channels	u8	1	-
5	bits	u8	1	-
6	sample_format	u8	1	-
7	reserved	u8	1	-
8	buffer	u32	4	-
12	prefill	u32	4	-
16	low	u32	4	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

45h STREAM_STATE

Читает состояние своего кольца. fill/free - байты; published/consumed/position - PCM-кадры; fraction - дробная часть Q0.32. Сверяйте next_sequence после потери ответа FEED. После CLOSE объект исчезает и запрос даёт HANDLE.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK: STREAM_STATE (70 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	state	u8	1	-
3	flags	u8	1	-
4	buffer	u32	4	-
8	prefill	u32	4	-
12	low	u32	4	-
16	fill	u32	4	-
20	free	u32	4	-
24	published	u64	8	-
32	consumed	u64	8	-
40	next_sequence	u32	4	-
44	underruns	u32	4	-
48	rate	u32	4	-
52	play_id	u16	2	-
54	channels	u8	1	-
55	bits	u8	1	-
56	error	u8	1	-
57	reserved	u8	1	-
58	fraction	u32	4	-
62	position	u64	8	-

46h STREAM_START

Однократно занимает основной звук и возвращает play_id. gain=0..64, pan=0..255. До prefill звучит тишина. Повторный START даёт STATE, нехватка голосов - NO_VOICE. Независимые FX сохраняются.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	gain	u8	1	-
3	pan	u8	1	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	play_id	u16	2	-

47h STREAM_FEED

Отправляет 1..1024 байта полных PCM-кадров; первый sequence=1, transport=00h/80h. Между запросом и ответом нужно обслужить WRITE_GRANT. WRITE_END не нужен. Неверный sequence даёт RANGE, выравнивание - LENGTH, нехватка места в кольце - NO_MEMORY. Успех возвращает следующий sequence и published в кадрах.

Запрос: STREAM_FEED_REQ (9 байт FAST; 13 байт VERIFIED).

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-
2	sequence	u32	4	-
6	length	u16	2	-
8	transport	u8	1	-
9	crc	u32	4	только VERIFIED

Ответ при OK: STREAM_FEED_RESP (12 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	next_sequence	u32	4	-
4	published	u64	8	-

48h STREAM_EOF

Сообщает, что новых данных не будет. Остаток доигрывается даже при fill меньше prefill. После этого FEED даёт STATE. Дождитесь state=3 и вызовите CLOSE: EOF сам по себе не освобождает кольцо.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK.

Нет полезных данных (0 байт).

49h STREAM_CLOSE

Закрывает указанное кольцо и освобождает его после применения события звуковой частью. Состояние closing можно увидеть в STATE. Повтор во время закрытия даёт STATE, после удаления - HANDLE; заполненная очередь событий - BUSY.

Запрос.

Смещ.	Поле	Тип	Байт	Примечание
0	handle	u16	2	-

Ответ при OK.

Нет полезных данных (0 байт).

50h SNAPSHOT

Фиксирует согласованную сводку и записи голосов/FX. Для страниц используйте возвращённые session и snapshot_seq. Новый SNAPSHOT или SNAP_LATCH заменяет удерживаемую копию. Сам запрос не останавливает звук.

Запрос.

Нет полезных данных (0 байт).

Ответ при OK: SNAPSHOT (64 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	snapshot_seq	u32	4	-
4	session	u64	8	-
12	output_frames	u64	8	-
20	play_id	u16	2	-
22	reserved20	u16	2	значение 0
24	state	u8	1	-
25	kind	u8	1	-
26	channels	u16	2	-
28	object	u16	2	-
30	reserved30	u16	2	значение 0
32	source_frame	u32	4	-
36	order	u16	2	-
38	row	u8	1	-
39	tick	u8	1	-
40	speed	u8	1	-
41	bpm	u8	1	-
42	peak_left	u16	2	-
44	peak_right	u16	2	-
46	audio_underruns	u32	4	-

Смещ.	Поле	Тип	Байт	Примечание
50	last_audio_error	u8	1	-
51	reserved51	u8	1	значение 0
52	voice_count	u16	2	-
54	fx_count	u16	2	-
56	transfer_id	u32	4	-
60	transfer_done	u16	2	-
62	transfer_length	u16	2	-

Разрешена во время передачи блока.

51h FX_STATUS

Читает записи FX из удерживаемого снимка. first - индекс записи, max_records=1..6; большее значение ограничивается вместимостью ответа, ноль даёт BAD_ARGUMENT. Stereo занимает одну запись с channels=2. Неверный ключ снимка даёт STATE, first>total - RANGE; first=total возвращает count=0.

Запрос: PAGE_REQ (16 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	session	u64	8	-
8	snapshot_seq	u32	4	-
12	first	u16	2	-
14	max_records	u16	2	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	header	PAGE_HEADER	20	-
20	records	FXREC[]	по count	count записей указанного типа

Разрешена во время передачи блока.

52h VOICE_STATUS

Читает все физические голоса, включая незанятые. first - индекс записи, max_records=1..5; большее значение ограничивается вместимостью ответа, ноль даёт BAD_ARGUMENT. Ключ берётся из SNAPSHOT. Неверный ключ даёт STATE, first>total - RANGE; first=total возвращает пустую страницу.

Запрос: PAGE_REQ (16 байт).

Смещ.	Поле	Тип	Байт	Примечание
0	session	u64	8	-
8	snapshot_seq	u32	4	-
12	first	u16	2	-
14	max_records	u16	2	-

Ответ при OK.

Смещ.	Поле	Тип	Байт	Примечание
0	header	PAGE_HEADER	20	-
20	records	VOICEREC[]	по count	count записей указанного типа

Разрешена во время передачи блока.

9.8. Вложенные структуры и записи

PCM_DESC (24 байт)

Смещ.	Поле	Тип	Байт	Примечание
0	rate	u32	4	-
4	channels	u8	1	-
5	bits	u8	1	-
6	sample_format	u8	1	0=U8 1=S8 2=S16LE
7	flags	u8	1	Бит 0: петля вперёд
8	data_offset	u32	4	-
12	data_length	u32	4	-
16	loop_start	u32	4	-
20	loop_end	u32	4	-

PAGE_HEADER (20 байт)

Смещ.	Поле	Тип	Байт	Примечание
0	session	u64	8	-
8	snapshot_seq	u32	4	-
12	first	u16	2	-
14	count	u16	2	-
16	total	u16	2	-
18	record_size	u16	2	-

VOICEREC (20 байт)

Смещ.	Поле	Тип	Байт	Примечание
0	voice_id	u16	2	-
2	instance_id	u16	2	-
4	handle	u16	2	-
6	logical_channel	u16	2	-
8	source_frame	u32	4	-
12	peak	u16	2	-
14	period	u16	2	-
16	note	u8	1	-
17	volume	u8	1	-
18	pan	u8	1	-
19	flags	u8	1	-

FXREC (16 байт)

Смещ.	Поле	Тип	Байт	Примечание
0	instance_id	u16	2	-
2	voice_count	u16	2	-
4	handle	u16	2	-
6	state	u8	1	0=неактивен, 1=играет, 2=пауза
7	channels	u8	1	-
8	source_frame	u32	4	-
12	gain	u8	1	-
13	pan	u8	1	-
14	priority	u8	1	-
15	flags	u8	1	Бит 0: подмена канала GS; бит 1: повтор всего объекта